TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP TP. HỒ CHÍ MINH

BÁO CÁO CUỐI KỲ

Môn học: Phân tích dữ liệu lớn

Đề tài 8

Phân tích dữ liệu IoT và bảo trì dự đoán

Phát hiện bất thường, dự đoán RUL và cảnh báo sớm trên dữ liệu cảm biến NASA C-MAPSS

Học viên thực hiện:	Phan Nguyễn Mai Phương
Mã số học viên:	25002711
Ngành:	Khoa học máy tính

TP. Hồ Chí Minh, tháng 10 năm 2026

Tóm tắt

Báo cáo thực hiện Đề tài 8 (Phân tích dữ liệu IoT và bảo trì dự đoán) trên bộ dữ liệu cảm biến mô phỏng NASA C-MAPSS, phân tích chính trên subset FD001 gồm 100 động cơ turbofan chạy đến khi hỏng. Mục tiêu là phát hiện dấu hiệu bất thường và dự đoán thời điểm cần bảo trì, kèm biểu đồ xu hướng cảnh báo sớm.

Dữ liệu được kiểm tra schema, khóa và tính liên tục của cycle, rồi lưu theo các lớp Bronze, Silver và Gold bằng Parquet và SQLite. ETL và rolling features được viết bằng cả Pandas và PySpark; hai implementation khớp hoàn toàn trên cùng input, và ở scale $10\times$, Spark local nhanh hơn Pandas khoảng 3,4 lần. Tín hiệu được khử nhiễu bằng EWMA span 10, giảm khoảng 86% dao động ngắn hạn mà vẫn giữ xu hướng suy giảm, rồi tổng hợp theo window 30 cycles thành 126 đặc trưng.

Kết quả gồm hai mức cảnh báo. Mức một dùng Health Index từ tín hiệu đã khử nhiễu: khi chỉ số giảm quá 3 độ lệch chuẩn so với 30 cycles đầu, cả 100/100 động cơ train được phát hiện bất thường trước khi hỏng, trung vị 121 cycles. Mức hai dùng HistGradientBoosting dự đoán RUL, đạt MAE 15,35 cycles (khoảng tin cậy 95%: 12,73–18,09) và RMSE 20,54 cycles trên 100 động cơ test. Với ngưỡng RUL dự đoán ≤ 30 cycles, không động cơ nào bị bỏ lỡ và lead time trung vị là 30 cycles, nhưng 49% cảnh báo bị trễ, tập trung ở động cơ suy giảm nhanh; ngưỡng 40 giảm tỷ lệ trễ xuống 6%. Ghép hai mức cảnh báo, báo cáo lập lịch dự đoán thời điểm cần bảo trì cho 100 động cơ test: 21 động cơ được báo cần bảo trì ngay và cả 21 đều đúng; 4 động cơ còn lại thật sự cần bảo trì được xếp đến hạn sau 1–8 cycles. Kết quả chỉ áp dụng cho dữ liệu mô phỏng; dữ liệu không có chi phí bảo trì nên báo cáo không đề xuất lịch bảo trì tối ưu.
Từ khóa: bảo trì dự đoán; IoT; NASA C-MAPSS; khử nhiễu EWMA; phát hiện bất thường; time window; RUL; cảnh báo sớm; lịch bảo trì; PySpark.

Mục lục

Tóm tắt i

Thuật ngữ và chữ viết tắt iv

1 Giới thiệu 1
1.1 Bối cảnh và bài toán . . . 1
1.2 Mục tiêu và câu hỏi nghiên cứu . . . 1
1.3 Phạm vi và cấu trúc báo cáo . . . 2

2 Dữ liệu 2
2.1 Nguồn dữ liệu . . . 2
2.2 Cấu trúc và ý nghĩa các cột . . . 3
2.3 Đặc điểm của tập train FD001 . . . 4
2.4 Kiểm tra chất lượng và cách xử lý . . . 5

3 Cơ sở lý thuyết 6
3.1 Dự đoán RUL và các nghiên cứu liên quan . . . 6
3.2 Metric đánh giá . . . 6
3.3 Trailing window và đặc trưng thống kê . . . 7
3.4 Khử nhiễu bằng làm mượt hàm mũ . . . 7
3.5 Kiểm định ghép cặp và hiệu chỉnh đa kiểm định . . . 7
3.6 Health Index, phát hiện bất thường và ARIMA . . . 8
3.7 Hadoop: HDFS và MapReduce . . . 8
3.8 Spark, Bronze–Silver–Gold và Parquet . . . 9

4 Tổ chức, truy vấn và biến đổi dữ liệu 10
4.1 Thu thập dữ liệu . . . 10
4.2 Gom dữ liệu từ nhiều file . . . 11
4.3 Lưu trữ theo lớp . . . 11
4.4 Lưu trữ phân tán trên HDFS . . . 12
4.5 Truy vấn dữ liệu . . . 14
4.6 Các bước biến đổi . . . 15
4.7 Cột được dùng cho mô hình . . . 17
4.8 Quan hệ giữa các bảng . . . 18

5 Phương pháp 20
5.1 Tổng quan pipeline . . . 20
5.2 Khử nhiễu và Health Index . . . 21
5.3 Phát hiện dấu hiệu bất thường . . . 22
5.4 Kiểm định xu hướng đầu/cuối vòng đời . . . 22
5.5 Huấn luyện và đánh giá mô hình RUL . . . 22
5.6 Cảnh báo sớm . . . 23
5.7 Báo cáo dự đoán thời điểm cần bảo trì . . . 23
5.8 Thử nghiệm mở rộng . . . 24

6 Kết quả **25**
6.1 Pipeline dữ liệu và chất lượng dữ liệu . . . 25
6.2 Xử lý nhiễu và tổng hợp theo cửa sổ thời gian . . . 26
6.3 Xu hướng suy giảm và dấu hiệu bất thường . . . 27
6.4 Dự đoán RUL . . . 29
6.5 Cảnh báo sớm . . . 31
6.6 Báo cáo dự đoán thời điểm cần bảo trì . . . 32
6.7 Thử nghiệm mở rộng . . . 34

7 Thảo luận **36**
7.1 Sai số của mô hình RUL đến từ đâu . . . 36
7.2 Vì sao gần một nửa cảnh báo bị trễ . . . 37
7.3 Vì sao mô hình FD001 không chuyển được sang subset khác . . . 38
7.4 Ý nghĩa đối với xử lý dữ liệu lớn . . . 38
7.5 Giới hạn . . . 39
7.6 Hướng phát triển . . . 39

8 Khó khăn và bài học rút ra **40**

9 Kết luận **41**

Tài liệu tham khảo **42**

Thuật ngữ và chữ viết tắt

Thuật ngữ	Ý nghĩa trong báo cáo
RUL	Remaining Useful Life, tuổi thọ hữu ích còn lại: số cycle động cơ còn vận hành được trước khi hỏng.
C-MAPSS	Commercial Modular Aero-Propulsion System Simulation, bộ dữ liệu mô phỏng động cơ turbofan của NASA, gồm bốn subset FD001–FD004.
Cycle	Một chu kỳ vận hành của động cơ; là đơn vị thời gian rời rạc của dữ liệu, không quy đổi được sang giờ hoặc ngày.
Train / test	Train chứa lịch sử vận hành đến khi hỏng; test bị cắt trước khi hỏng và có file nhãn RUL riêng.
Trailing window	Đoạn w cycles liên tiếp gần nhất của một động cơ tính đến cycle đang xét; báo cáo dùng $w = 30$.
HDFS	Hadoop Distributed File System, hệ thống file phân tán: chia file thành block 128 MB lưu trên DataNode; NameNode giữ metadata.
MapReduce	Mô hình tính toán hai pha: map sinh cặp khóa–giá trị, reduce tổng hợp các giá trị cùng khóa; chạy trên YARN.
YARN	Thành phần quản lý tài nguyên của Hadoop, cấp phát và chạy các task map, reduce trên cụm.
EWMA	Exponentially weighted moving average, trung bình trượt có trọng số mũ; báo cáo dùng span 10 để khử nhiễu sensor.
GroupKFold	Cross-validation chia fold theo nhóm; ở đây mỗi nhóm là một động cơ, nên mọi window của một động cơ nằm cùng một fold.
OOF	Out-of-fold: dự đoán cho một động cơ được tạo bởi mô hình không dùng động cơ đó khi huấn luyện.
Nested CV	Cross-validation lồng nhau: vòng trong chọn cấu hình, vòng ngoài đánh giá cả quy trình chọn.
MAE / RMSE	Sai số tuyệt đối trung bình / căn bậc hai của sai số bình phương trung bình, tính bằng cycle.
NASA score	Điểm phạt bất đối xứng, phạt dự đoán trễ (cao hơn RUL thật) nặng hơn dự đoán sớm.
CI	Confidence interval, khoảng tin cậy.
HGB	HistGradientBoostingRegressor của scikit-learn.
Data leakage	Rò rỉ dữ liệu: thông tin không có tại thời điểm dự đoán (như nhãn test hoặc cycle tương lai) lọt vào quá trình huấn luyện hoặc chọn mô hình.
EDA	Exploratory Data Analysis, phân tích khám phá dữ liệu.
ETL	Extract, Transform, Load: đọc, biến đổi và ghi dữ liệu.
Bronze / Silver / Gold	Ba lớp lưu trữ: dữ liệu gốc; dữ liệu đã kiểm tra và chuẩn hóa kiểu; bảng tổng hợp phục vụ phân tích.

Tiếp tục ở trang sau

Thuật ngữ	Ý nghĩa trong báo cáo
Parquet / SQLite	Định dạng lưu trữ dạng cột / cơ sở dữ liệu quan hệ dạng file.
Parity	Đối chiếu kết quả của hai implementation (Pandas và PySpark) trên cùng input.
Lead time	Thời gian báo trước: số cycle từ cảnh báo đầu tiên đến khi động cơ hỏng.
Health Index	Chỉ số tổng hợp trạng thái động cơ, ở đây là thành phần chính thứ nhất của 21 sensor đã chuẩn hóa (14 sensor có xu hướng được khử nhiễu trước).
Cảnh báo mức 1 / mức 2	Mức 1: Health Index giảm quá 3 độ lệch chuẩn so với 30 cycles đầu trong 5 cycles liên tiếp (dấu hiệu bất thường). Mức 2: RUL dự đoán không vượt quá 30 cycles (cần bảo trì).
Lịch bảo trì	Bảng cho mỗi động cơ test: cycle hiện tại, RUL dự đoán, cycle cần bảo trì (lúc RUL dự đoán chạm 30) và trạng thái.
PCA	Principal Component Analysis, phân tích thành phần chính.
ARIMA	Mô hình chuỗi thời gian tự hồi quy, sai phân và trung bình trượt.
AIC	Akaike Information Criterion, tiêu chí chọn bậc mô hình.
ADF	Kiểm định Augmented Dickey–Fuller về nghiệm đơn vị (tính dừng).
Ljung–Box	Kiểm định tự tương quan của phần dư.
Wilcoxon	Kiểm định dấu-hạng Wilcoxon cho dữ liệu ghép cặp.
BH / FDR	Hiệu chỉnh Benjamini–Hochberg nhằm kiểm soát False Discovery Rate (tỷ lệ phát hiện sai).

Danh mục hình

1 Luồng dữ liệu C-MAPSS . . . 10
2 HDFS và YARN . . . 14
3 Minh họa window và nhãn . . . 17
4 Quan hệ giữa các bảng . . . 18
5 Pipeline xử lý dữ liệu . . . 20
6 Tín hiệu trước và sau khử nhiễu . . . 26
7 Tương quan sensor–RUL . . . 27
8 Phát hiện bất thường bằng Health Index . . . 28
9 RUL thật và dự đoán test . . . 30
10 Sai số theo RUL test . . . 31
11 Quỹ đạo cảnh báo . . . 32
12 Trạng thái đội động cơ test . . . 33
13 Kế hoạch bảo trì theo thời gian . . . 33
14 RUL OOF so với thật . . . 36
15 Cảnh báo theo tuổi thọ . . . 37

Danh mục bảng

2 Quy mô C-MAPSS . . . 2
3 Từ điển dữ liệu FD001 . . . 3
4 Nhóm cột train FD001 . . . 4
5 Hồ sơ sensor FD001 . . . 4
6 Chất lượng dữ liệu . . . 5
7 Vị trí nghiên cứu . . . 6
8 Các bảng FD001 . . . 12
9 Kết quả bước HDFS . . . 13
10 Các bước biến đổi FD001 . . . 16
11 Vai trò nhóm cột . . . 18
12 Kiểm tra quan hệ bảng . . . 19
13 Công cụ và đầu ra . . . 21
14 Benchmark pipeline local . . . 25
15 Tuổi thọ train FD001 . . . 26
16 Mức giảm dao động sau EWMA . . . 26
17 Thay đổi sensor . . . 28
18 Độ nhạy của quy tắc phát hiện bất thường . . . 29
19 So sánh mô hình GroupKFold . . . 30
20 Metric test của HGB . . . 30
21 Phân phối lead time . . . 31
22 Độ nhạy ngưỡng cảnh báo . . . 31

23 Lịch bảo trì dự đoán 32
24 Kiểm tra lịch bảo trì bằng nhãn test 33
25 Đánh giá chuyển miền 34
26 Sai số OOF theo RUL 36
27 Phạm vi Big Data theo 5V 39

1 Giới thiệu

1.1 Bối cảnh và bài toán

Bảo trì dự đoán cần trả lời hai câu hỏi: thiết bị đã có dấu hiệu bất thường chưa, và còn vận hành được bao lâu trước khi hỏng. Câu hỏi thứ hai được đo bằng Remaining Useful Life (RUL, tuổi thọ hữu ích còn lại). Nếu trả lời được hai câu hỏi này từ dữ liệu cảm biến, đội vận hành có thể lên kế hoạch can thiệp trước khi hỏng, thay vì bảo trì theo lịch cố định hoặc sửa chữa sau khi hỏng.

Báo cáo xem dự đoán RUL là bài toán hồi quy có giám sát trên chuỗi thời gian cảm biến, còn phát hiện bất thường là bài toán không dùng nhãn. Tại cycle t, mô hình chỉ được dùng dữ liệu đã quan sát đến cycle t; biến mục tiêu là số cycle còn lại trước khi động cơ hỏng. Trong tập train, mỗi động cơ được theo dõi đến khi hỏng, nên nhãn RUL được tính trực tiếp:

$$y_{i,t} = T_i - t, \qquad 1 \le t \le T_i, \tag{1}$$

trong đó T_i là cycle cuối cùng (cycle hỏng) của động cơ i và $y_{i,t}$ là RUL tại cycle t. Ví dụ, một động cơ hỏng ở cycle 200 thì tại cycle 170 có RUL bằng 30. Công thức (1) không cắt ngưỡng RUL, nên nhãn ở đầu vòng đời có thể lên tới hàng trăm cycles.

Tập test khác ở chỗ chuỗi bị cắt tại một thời điểm trước khi hỏng. NASA cung cấp riêng RUL thật tại cycle cuối được quan sát của mỗi động cơ test. Vì vậy, đánh giá cuối gồm đúng một dự đoán cho mỗi động cơ test, tại cycle cuối của nó.

1.2 Mục tiêu và câu hỏi nghiên cứu

Báo cáo thực hiện Đề tài 8 của môn Phân tích dữ liệu lớn: phân tích dữ liệu cảm biến thiết bị để phát hiện dấu hiệu bất thường và dự đoán thời điểm cần bảo trì. Mục tiêu là xây dựng một pipeline tái lập được, đi qua đủ các bước thu thập, làm sạch, chuẩn hóa, tích hợp, lưu trữ và trực quan hóa, rồi tạo ra sản phẩm cuối là dự đoán thời điểm cần bảo trì kèm biểu đồ xu hướng cảnh báo sớm. Năm câu hỏi được sắp theo đúng thứ tự pipeline:

- **Q1.** Dữ liệu có hợp lệ về schema, khóa và tính liên tục của cycle không, và ETL bằng Pandas và PySpark có cho kết quả giống nhau trên cùng input không?
- **Q2.** Nhiễu tín hiệu cảm biến được giảm bao nhiêu sau khử nhiễu, và dữ liệu được tổng hợp theo cửa sổ thời gian như thế nào?
- **Q3.** Sensor suy giảm theo xu hướng nào, và có phát hiện được dấu hiệu bất thường trước khi động cơ hỏng không?
- **Q4.** Dự đoán RUL đạt độ chính xác bao nhiêu trên tập test độc lập?
- **Q5.** Dự đoán RUL và tín hiệu bất thường có thể chuyển thành cảnh báo sớm và lịch bảo trì cho từng động cơ với chất lượng ra sao?

1.3 Phạm vi và cấu trúc báo cáo

C-MAPSS là dữ liệu mô phỏng, nên mọi kết luận chỉ áp dụng trong môi trường mô phỏng này. Dữ liệu cũng không có chi phí bảo trì, thời gian ngừng máy hay cách quy đổi cycle sang giờ. Vì vậy, lịch bảo trì trong báo cáo dùng một ngưỡng RUL cố định và được tính bằng cycle; báo cáo không tối ưu lịch theo chi phí.

Khai báo hỗ trợ công cụ. Trong quá trình thực hiện báo cáo, công cụ AI/LLM được sử dụng để gợi ý cấu trúc, hỗ trợ viết và rà soát một số đoạn mã, notebook, kịch bản kiểm tra và văn bản. Công cụ không được dùng để bịa số liệu, thay thế việc chạy thực nghiệm hoặc tự quyết định kết luận. Người thực hiện trực tiếp kiểm tra mã nguồn, dữ liệu, artifact, trích dẫn và nội dung cuối cùng, đồng thời chịu trách nhiệm về toàn bộ kết quả được trình bày.

Phần còn lại được tổ chức như sau. Mục 2 mô tả dữ liệu và kiểm tra chất lượng. Mục 3 trình bày cơ sở lý thuyết. Mục 4 mô tả cách gom, lưu trữ, truy vấn và biến đổi dữ liệu thành đặc trưng. Mục 5 mô tả khử nhiễu, phát hiện bất thường, mô hình và cảnh báo. Mục 6 trình bày kết quả, Mục 7 thảo luận và nêu giới hạn, Mục 8 nêu khó khăn và bài học, Mục 9 kết luận. Phân tích chính dùng FD001; ba subset FD002–FD004 chỉ dùng trong thử nghiệm mở rộng.

2 Dữ liệu

2.1 Nguồn dữ liệu

Đề tài gợi ý hai nguồn: bộ Predictive Maintenance trên Kaggle và bộ NASA Turbofan Engine. Bài làm chọn NASA C-MAPSS vì mỗi động cơ có chuỗi cảm biến theo cycle cho tới khi hỏng, và tập test có nhãn RUL thật để đánh giá. Bộ Kaggle là bảng phân loại lỗi, mỗi dòng là một lần đo độc lập, không có chuỗi chạy đến khi hỏng của từng máy nên không phù hợp với yêu cầu xử lý chuỗi thời gian và dự đoán thời điểm bảo trì.

NASA C-MAPSS mô phỏng các chuỗi vận hành đến khi hỏng của một đội động cơ turbofan cùng loại [10, 13]. Mỗi động cơ bắt đầu với mức hao mòn ban đầu khác nhau, vận hành bình thường, rồi phát sinh lỗi và suy giảm dần đến khi hỏng. Bộ dữ liệu gồm bốn subset khác nhau về số điều kiện vận hành và số dạng lỗi (Bảng 2).

Bảng 2: Quy mô và đặc điểm của bốn subset C-MAPSS.

Subset	Dòng train	Dòng test	Động cơ train	Động cơ test	Điều kiện	Dạng lỗi
FD001	20.631	13.096	100	100	1	1 (HPC)
FD002	53.759	33.991	260	259	6	1 (HPC)
FD003	24.720	16.596	100	100	1	2 (HPC, fan)
FD004	61.249	41.214	249[a]	248[a]	6	2 (HPC, fan)

HPC: High-Pressure Compressor (máy nén áp suất cao). [a]Số động cơ đếm từ file dữ liệu; file `readme.txt` của NASA ghi ngược thành 248 train và 249 test.

FD001 được chọn làm subset chính vì chỉ có một điều kiện vận hành và một dạng lỗi. Khi

các yếu tố này cố định, biến động của sensor chủ yếu phản ánh quá trình suy giảm, giúp việc diễn giải từng bước phân tích rõ ràng hơn. FD002–FD004 chỉ dùng trong thử nghiệm mở rộng, để kiểm tra mô hình FD001 khi điều kiện vận hành và dạng lỗi thay đổi.

2.2 Cấu trúc và ý nghĩa các cột

Mỗi dòng trong file train và test là một cycle của một động cơ, gồm 26 giá trị cách nhau bởi khoảng trắng: mã động cơ, số thứ tự cycle, ba operational settings và 21 sensor. File gốc không có header; tên cột trong Bảng 3 do repository đặt. NASA chỉ gọi các cột sensor là "sensor measurement" mà không cho biết tên đại lượng vật lý hay đơn vị, vì vậy báo cáo giữ tên `sensor_1` đến `sensor_21` và không suy đoán sensor nào đo nhiệt độ hay áp suất.

Bảng 3: Từ điển dữ liệu của FD001.

Cột	Ý nghĩa	Kiểu và miền giá trị	Cách sử dụng
`split`	Cột thêm khi đọc file, cho biết dòng thuộc train hay test.	`train` hoặc `test`.	Thành phần của khóa; giữ tách biệt hai tập.
`engine_id`	Mã động cơ trong từng tập.	Số nguyên 1–100 ở cả train và test.	Đơn vị chia fold, ghép cặp và tính metric.
`cycle`	Thứ tự cycle của động cơ.	Số nguyên, bắt đầu từ 1, liên tục.	Trục thời gian; dùng để tạo nhãn và window.
`setting_1`–`setting_3`	Ba operational settings.	`float64`; `setting_3` hằng bằng 100 trên train.	Không đưa vào mô hình RUL hiện tại.
`sensor_1`–`sensor_21`	Giá trị đo của 21 sensor trong cycle.	`float64`, không thiếu giá trị.	Nguồn tín hiệu cho EDA, kiểm định, khử nhiễu, Health Index và đặc trưng.
`rul`	Nhãn RUL train tính theo Công thức (1).	Số nguyên 0–361.	Biến mục tiêu; không có trong file gốc.
`RUL_FD001.txt`	RUL thật tại cycle cuối của mỗi động cơ test.	100 số nguyên; dòng i ứng với động cơ test i.	Chỉ đọc ở bước đánh giá cuối.

Ví dụ, dòng đầu tiên của `train_FD001.txt` có `engine_id=1`, `cycle=1`, ba settings lần lượt là $-0{,}0007$, $-0{,}0004$ và $100{,}0$, tiếp theo là 21 giá trị sensor. Động cơ này hỏng ở cycle 192, nên dòng đầu có `rul=191`.

Hai động cơ có cùng `engine_id` ở train và test là hai động cơ khác nhau. Vì vậy, khóa của một dòng là bộ ba `(split, engine_id, cycle)`, không phải cặp `(engine_id, cycle)`.

2.3 Đặc điểm của tập train FD001

Bảng 4 tóm tắt tập train sau khi gắn nhãn RUL. Không có giá trị thiếu và không có khóa trùng. Điểm đáng chú ý là sáu sensor hằng số trên toàn bộ train, còn `sensor_6` chỉ có hai giá trị và không đổi trong 38/100 động cơ. Chi tiết từng sensor nằm ở Bảng 5.

Bảng 4: Tóm tắt các nhóm cột trên train FD001.

Nhóm cột	Số cột	Thiếu	Nhận xét
`engine_id`, `cycle`	2	0	20.631 dòng của 100 động cơ; khóa không trùng, cycle liên tục.
Operational settings	3	0	`setting_1` có 158 giá trị, `setting_2` có 13 giá trị, `setting_3` hằng bằng 100.
Sensor	21	0	Sáu sensor hằng; `sensor_6` hằng trong 38 động cơ.
Nhãn `rul`	1	0	Từ 0 đến 361 cycles; trung vị theo dòng là 103 cycles.

Bảng 5: Số giá trị khác nhau và miền giá trị của 21 sensor trên train FD001.

Sensor	Số giá trị	Min–max	Ghi chú
`sensor_1`	1	518,67–518,67	Hằng.
`sensor_2`	310	641,21–644,53	
`sensor_3`	3.012	1.571,04–1.616,91	
`sensor_4`	4.051	1.382,25–1.441,49	Tăng rõ ở cuối vòng đời.
`sensor_5`	1	14,62–14,62	Hằng.
`sensor_6`	2	21,60–21,61	Hằng trong 38/100 động cơ.
`sensor_7`	513	549,85–556,06	
`sensor_8`	53	2.387,90–2.388,56	Biến thiên hẹp.
`sensor_9`	6.403	9.021,73–9.244,59	Xu hướng khác nhau giữa các động cơ.
`sensor_10`	1	1,30–1,30	Hằng.
`sensor_11`	159	46,85–48,53	Tương quan Spearman âm mạnh với RUL.
`sensor_12`	427	518,69–523,38	Tương quan Spearman dương mạnh với RUL.
`sensor_13`	56	2.387,88–2.388,56	Biến thiên hẹp.
`sensor_14`	6.078	8.099,94–8.293,72	Xu hướng khác nhau giữa các động cơ.
`sensor_15`	1.918	8,3249–8,5848	
`sensor_16`	1	0,03–0,03	Hằng.
`sensor_17`	13	388–400	Giá trị rời rạc.
`sensor_18`	1	2.388–2.388	Hằng.
`sensor_19`	1	100–100	Hằng.
`sensor_20`	120	38,14–39,43	
`sensor_21`	4.745	22,8942–23,6184	

Sensor hằng không mang thông tin về suy giảm và không có hệ số tương quan xác định.

Báo cáo vẫn giữ đủ 21 sensor trong các bảng thống kê để kết quả phản ánh toàn bộ dữ liệu, thay vì chỉ những sensor cho kết quả đẹp.

2.4 Kiểm tra chất lượng và cách xử lý

Trước mọi phân tích, bộ đọc dữ liệu kiểm tra: mỗi dòng có đúng 26 trường, mọi giá trị là số hữu hạn, khóa không trùng, cycle bắt đầu từ 1 và liên tục trong từng động cơ. Với nhãn train, bộ kiểm tra đối chiếu lại Công thức (1). Khi phát hiện lỗi, chương trình dừng với `ContractError` thay vì tự điền hay xóa dòng. Các kiểm tra này quan trọng vì dữ liệu sai vẫn có thể cho ra thống kê trông hợp lý: chẳng hạn chuỗi cycle $1, 2, 4$ không được coi là ba bước liên tiếp khi tạo window. Mỗi lần chạy lưu mã băm SHA-256 của file đầu vào để nhận biết phiên bản dữ liệu đã dùng.

Bảng 6 tóm tắt các đặc điểm dữ liệu cần lưu ý và cách báo cáo xử lý.

Bảng 6: Đặc điểm dữ liệu và cách xử lý.

Đặc điểm	Ảnh hưởng	Cách xử lý
Không thiếu giá trị, khóa hợp lệ	Không cần điền dữ liệu.	Giữ nguyên dữ liệu; dừng nếu kiểm tra thất bại.
Độ dài chuỗi khác nhau (128–362 cycles)	Động cơ sống lâu đóng góp nhiều window hơn.	Chia fold, gán trọng số và tính metric theo động cơ.
Sensor hằng hoặc ít biến thiên	Tương quan và kiểm định không xác định.	Giữ trong bảng, đánh dấu "không xác định".
Nhiễu cảm biến	Dao động giữa hai cycle liền kề che khuất xu hướng suy giảm.	Làm mượt bằng EWMA chỉ dùng quá khứ (Mục 5.2); không xóa hoặc thay giá trị gốc.
Không có đơn vị vật lý	Không diễn giải được ý nghĩa vật lý của sensor.	Dùng tên ẩn danh; chênh lệch hiểu theo đơn vị gốc.
Nhãn test tách riêng	Đọc sớm sẽ gây data leakage.	Chỉ đọc `RUL_FD001.txt` sau khi đã ghi dự đoán test.

3 Cơ sở lý thuyết

3.1 Dự đoán RUL và các nghiên cứu liên quan

Prognostics là lĩnh vực nghiên cứu diễn biến suy giảm của hệ thống và thời gian còn lại trước khi hỏng. Saxena và cộng sự xây dựng C-MAPSS để có dữ liệu run-to-failure với nhãn đã biết cho bài toán này [13]. Các nghiên cứu sau đó tiếp cận theo hai hướng chính: trích xuất đặc trưng thủ công rồi dùng mô hình hồi quy, hoặc học trực tiếp từ chuỗi bằng deep learning như CNN và LSTM [8, 9, 24]. Ramasso và Saxena lưu ý rằng kết quả trên C-MAPSS phụ thuộc mạnh vào subset và cách đánh giá [11]; vì vậy, các con số giữa những bài báo dùng cách tiền xử lý hoặc chia dữ liệu khác nhau không so sánh trực tiếp được.

Báo cáo chọn hướng thứ nhất. Đặc trưng thống kê trên window kết hợp với mô hình cây cho phép kiểm tra từng bước của pipeline và phù hợp với trọng tâm của môn học là xử lý dữ liệu tái lập được. Bảng 7 tóm tắt vị trí của báo cáo so với các nguồn tham khảo chính.

Bảng 7: Vị trí của báo cáo so với một số nghiên cứu liên quan.

Nguồn	Nội dung	Liên hệ với báo cáo
Saxena và cộng sự [13]	Mô phỏng suy giảm, tạo C-MAPSS	Cơ sở để hiểu dữ liệu và cách tạo nhãn RUL.
Ramasso và Saxena [11]	Tổng quan các phương pháp trên C-MAPSS	Lý do báo cáo mô tả rõ subset, cách chia dữ liệu và metric.
Lei và cộng sự [8]	Tổng quan machinery prognostics	Bối cảnh chung từ thu thập dữ liệu đến dự đoán RUL.
Li và cộng sự [9]	CNN trên window thời gian	Tham khảo độ dài window 30 cycles và NASA score.
Zheng và cộng sự [24]	LSTM cho dự đoán RUL	Hướng deep learning chưa được thực hiện trong báo cáo.
Saxena và cộng sự [12]	Metric đánh giá prognostics	Phân biệt đánh giá tại một điểm cắt với đánh giá trên toàn chuỗi.

3.2 Metric đánh giá

Với n động cơ, sai số $e_i = \hat{y}_i - y_i$ là RUL dự đoán trừ RUL thật. MAE bằng $\frac{1}{n}\sum |e_i|$ và RMSE bằng $\sqrt{\frac{1}{n}\sum e_i^2}$; cả hai có đơn vị cycle, trong đó RMSE nhạy hơn với các sai số lớn. NASA score cộng điểm phạt của từng dự đoán [9]:

$$S = \sum_{i=1}^{n} s(e_i), \qquad s(e) = \begin{cases} \exp(-e/13) - 1, & e < 0, \\ \exp(e/10) - 1, & e \geq 0. \end{cases} \tag{2}$$

Mẫu số 10 nhỏ hơn 13, nên dự đoán cao hơn RUL thật (báo hỏng muộn) bị phạt nặng hơn dự đoán thấp hơn cùng độ lớn, phản ánh việc báo muộn có thể khiến động cơ hỏng trước khi kịp bảo trì [13]. NASA score không có đơn vị và tăng theo hàm mũ, nên chỉ dùng để so sánh các mô hình trên cùng tập đánh giá.

Theo Saxena và cộng sự [12], một dự đoán tại cycle cuối của mỗi động cơ test chỉ đo chất lượng tại điểm cắt đó, không đo sai số dọc theo toàn bộ vòng đời.

3.3 Trailing window và đặc trưng thống kê

Tại cycle t, mẫu đầu vào của động cơ i là ma trận gồm w cycles gần nhất:

$$W_{i,t} = [\mathbf{x}_{i,t-w+1}, \ldots, \mathbf{x}_{i,t}]^{\mathsf{T}} \in \mathbb{R}^{w \times J}, \qquad t \geq w, \tag{3}$$

trong đó $\mathbf{x}_{i,t}$ là vector $J = 21$ giá trị sensor tại cycle t. Báo cáo dùng $w = 30$, một độ dài từng được sử dụng cho FD001 [9]. Window chỉ chứa quá khứ và hiện tại, nên không có thông tin từ cycle tương lai; ví dụ, mẫu tại cycle 170 dùng cycles 141–170.

Mỗi cột sensor trong window được tóm tắt bằng sáu thống kê: giá trị cuối, trung bình, độ lệch chuẩn mẫu, min, max và độ dốc tuyến tính. Với dãy $z_1, \ldots, z_w$, độ dốc là hệ số của hồi quy bình phương tối thiểu theo vị trí k [7]:

$$b = \frac{\sum_{k=1}^{w}(k - \bar{k})(z_k - \bar{z})}{\sum_{k=1}^{w}(k - \bar{k})^2}, \tag{4}$$

với $\bar{k}$ và $\bar{z}$ là trung bình của vị trí và của giá trị sensor. Đơn vị của b là đơn vị sensor trên một cycle. Giá trị cuối và trung bình cho biết mức tín hiệu, còn độ dốc cho biết tín hiệu đang tăng hay giảm trong 30 cycles gần nhất.

3.4 Khử nhiễu bằng làm mượt hàm mũ

EWMA (exponentially weighted moving average, trung bình trượt có trọng số mũ) là cách làm mượt đơn giản nhất trong họ exponential smoothing [7]. Giá trị làm mượt tại cycle t là trung bình có trọng số của giá trị hiện tại và giá trị làm mượt trước đó:

$$s_t = \alpha x_t + (1 - \alpha) s_{t-1} = \alpha \sum_{k=0}^{t-2} (1 - \alpha)^k x_{t-k} + (1 - \alpha)^{t-1} x_1, \tag{5}$$

với $0 < \alpha \leq 1$. Trọng số của các quan sát giảm theo cấp số nhân khi lùi về quá khứ, nên dao động ngắn hạn bị triệt tiêu một phần, còn xu hướng chậm vẫn được giữ. α càng nhỏ thì đường càng mượt nhưng càng phản ứng chậm. EWMA chỉ dùng quá khứ, nên áp dụng được tại thời điểm dự đoán mà không rò rỉ thông tin tương lai.

3.5 Kiểm định ghép cặp và hiệu chỉnh đa kiểm định

Khi so sánh một sensor ở đầu và cuối vòng đời của cùng một động cơ, phép ghép cặp loại bỏ khác biệt nền giữa các động cơ. Kiểm định dấu-hạng Wilcoxon xếp hạng độ lớn các chênh lệch rồi so sánh tổng hạng của chênh lệch dương và âm; dưới giả định phân phối chênh lệch đối xứng, nó kiểm định trung vị chênh lệch bằng 0 [22, 19]. Bootstrap lấy mẫu lại có hoàn lại các

động cơ để ước lượng khoảng tin cậy cho trung bình chênh lệch mà không cần giả định phân phối chuẩn [6, 17].

Khi kiểm định đồng thời 21 sensor, xác suất có ít nhất một kết quả dương tính giả tăng lên. Thủ tục Benjamini–Hochberg điều chỉnh các p-value để kiểm soát tỷ lệ phát hiện sai kỳ vọng (FDR) trong nhóm kiểm định [4, 18].

3.6 Health Index, phát hiện bất thường và ARIMA

Health Index là một chỉ số duy nhất tóm tắt trạng thái của động cơ từ nhiều sensor. Báo cáo tạo Health Index bằng cách chuẩn hóa 21 sensor rồi lấy thành phần chính thứ nhất (PC1) của PCA [15]. PC1 là hướng có phương sai lớn nhất; trong FD001, phần lớn phương sai đến từ xu hướng suy giảm chung của các sensor. Dấu của PC1 được chọn sao cho chỉ số giảm dần theo cycle, tức giá trị thấp ứng với trạng thái xấu hơn.

Khi không có nhãn bất thường, một cách phổ biến là định nghĩa trạng thái bình thường từ chính dữ liệu ở giai đoạn vận hành ổn định, rồi báo khi giá trị lệch khỏi trạng thái đó quá một số lần độ lệch chuẩn. Ngưỡng ba độ lệch chuẩn tương ứng với xác suất rất nhỏ nếu dao động là ngẫu nhiên quanh mức tham chiếu; yêu cầu vượt ngưỡng nhiều cycle liên tiếp làm giảm thêm cảnh báo sinh ra từ một giá trị ngoại lai đơn lẻ. Cách làm này đo được thời điểm phát hiện, nhưng không đo được tỷ lệ báo động giả nếu không có dữ liệu động cơ vận hành bình thường đến hết chuỗi quan sát.

Chuỗi Health Index h_t của từng động cơ được mô hình hóa bằng ARIMA(p, d, q) [7]:

$$\phi(B)(1 - B)^d h_t = c + \theta(B)\varepsilon_t, \tag{6}$$

trong đó B là toán tử trễ, $\phi(B)$ và $\theta(B)$ là đa thức tự hồi quy bậc p và trung bình trượt bậc q, d là bậc sai phân và ε_t là nhiễu trắng. Kiểm định ADF đánh giá tính dừng của chuỗi [21], còn kiểm định Ljung–Box kiểm tra phần dư sau khi fit còn tự tương quan hay không.

3.7 Hadoop: HDFS và MapReduce

HDFS (Hadoop Distributed File System, hệ thống file phân tán của Hadoop) chia mỗi file thành các block lớn, mặc định 128 MB, và lưu các block trên DataNode; NameNode chỉ giữ metadata, tức file nào gồm những block nào và block nằm ở đâu [20]. Mỗi block có thể được nhân bản trên nhiều DataNode để chịu lỗi; số bản sao gọi là replication factor. MapReduce chia một phép tính thành hai pha [5]. Pha map đọc từng dòng và sinh cặp khóa–giá trị; pha reduce nhận mọi giá trị cùng một khóa, sau bước sắp xếp và chuyển dữ liệu (shuffle), để tổng hợp. Ví dụ, để tính tuổi thọ từng động cơ, map biến mỗi dòng quan sát thành cặp (động cơ, cycle), còn reduce lấy cycle lớn nhất của mỗi động cơ. YARN cấp phát tài nguyên và chạy các task map, reduce trên cụm. Hadoop Streaming cho phép viết mapper và reducer bằng bất kỳ chương trình nào đọc stdin và ghi stdout, ở đây là Python [2]. Spark có thể dùng chính HDFS làm nơi lưu trữ, nhưng giữ dữ liệu trung gian trong bộ nhớ thay vì ghi xuống đĩa sau mỗi pha như MapReduce [23].

3.8 Spark, Bronze–Silver–Gold và Parquet

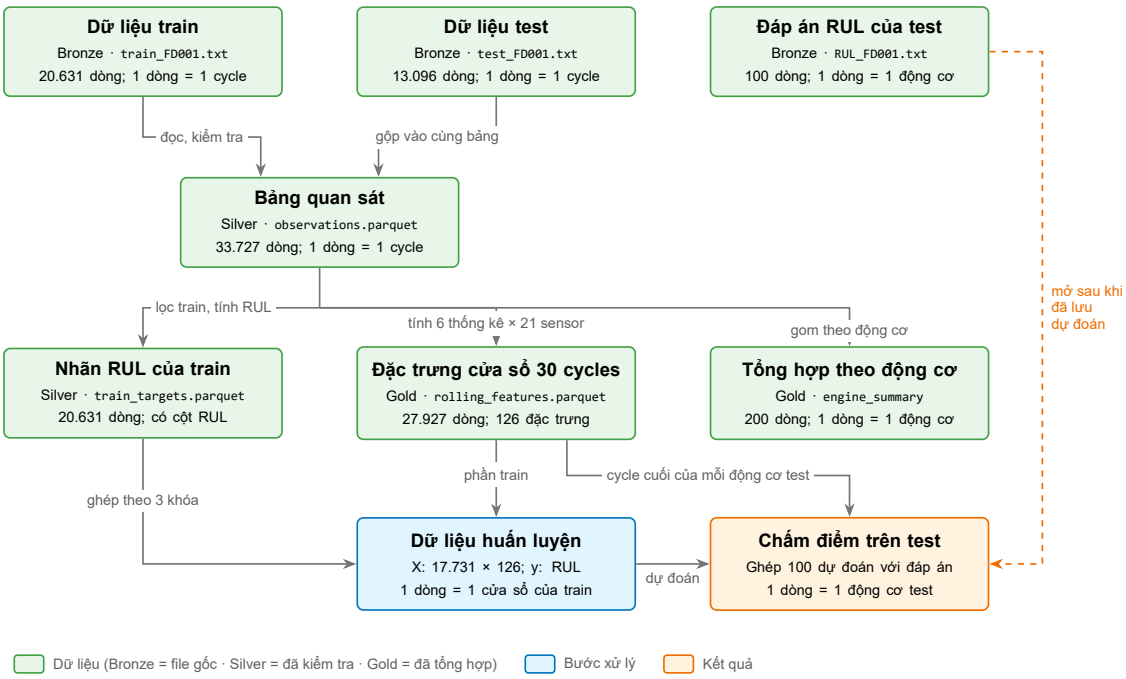
Pipeline dùng PySpark ở chế độ `local[4]` để thực hiện các phép biến đổi trên dữ liệu có schema. DataFrame API hỗ trợ `unionByName` để gom các file quan sát, `groupBy` và `agg` để tạo bảng tóm tắt theo động cơ, `join` để tạo nhãn RUL, cùng các phép sắp xếp theo `(split, engine_id, cycle)` [3]. Các thao tác này được thực thi lại trên cùng input với pipeline Pandas; kết quả được thu về để kiểm tra schema, khóa, số dòng và giá trị trong phép đối chiếu parity. Phần modeling vẫn dùng Pandas và scikit-learn, nên phạm vi Spark được giữ ở ETL và feature pipeline.

Tổ chức Bronze–Silver–Gold tách ba vai trò dữ liệu. Bronze giữ file text gốc và hash nguồn; Silver giữ observations đã kiểm tra kiểu dữ liệu cùng train targets; Gold giữ `engine_summary`, rolling features và các bảng phục vụ truy vấn. Parquet là định dạng lưu trữ theo cột, có encoding và compression cho dữ liệu phân tích [1]; SQLite giữ các bảng nhỏ có index để truy vấn nhanh. Cách phân lớp này giúp mỗi bước có input, output và kiểm tra riêng, đồng thời cho phép thay implementation ETL mà vẫn giữ contract dữ liệu.

4 Tổ chức, truy vấn và biến đổi dữ liệu

Mục này mô tả cách dữ liệu đi từ các file text gốc đến bảng đầu vào của mô hình, theo các câu hỏi về cách gom nhiều file, lưu trữ, truy vấn và biến đổi dữ liệu, qua những bước nào, dùng những cột nào, và các bảng quan hệ với nhau ra sao. Hình 1 tóm tắt toàn bộ luồng; mỗi hộp là một bảng, ghi rõ một dòng biểu diễn điều gì và kích thước thực tế với FD001. Mọi ví dụ trong mục này có thể chạy lại từng bước trong notebook `notebooks/00_data_pipeline_walkthrough.ipynb`; notebook gọi đúng các hàm của pipeline và in kết quả trung gian.

Theo vòng đời quản lý dữ liệu của môn học, pipeline được ánh xạ trực tiếp thành năm bước: **thu thập** gói C-MAPSS từ cổng dữ liệu mở của NASA bằng script, kiểm tra MD5 và SHA-256 rồi ghi manifest; **làm sạch** bằng schema contract, kiểm tra khóa, cycle và giá trị hữu hạn, rồi khử nhiễu sensor bằng EWMA; **chuẩn hóa** kiểu dữ liệu, tên cột, thứ tự khóa, và thang đo bằng StandardScaler trước khi tạo Health Index; **tích hợp** train/test bằng UNION, ghép nhãn theo khóa ba thành phần và tạo các bảng Silver–Gold; cuối cùng **lưu trữ** thành Parquet và SQLite để truy vấn bằng Pandas, SQL hoặc PySpark, đồng thời đưa lên HDFS để xử lý bằng MapReduce và Spark (Mục 4.4). Như vậy, mỗi bước đều có đầu vào, đầu ra và kiểm tra tương ứng thay vì chỉ mô tả bước tạo đặc trưng.



Hình 1: Luồng dữ liệu từ ba file gốc đến dữ liệu huấn luyện và bước chấm điểm. Chữ trên mũi tên là phép biến đổi; đáp án test chỉ được mở sau khi dự đoán đã được lưu.

4.1 Thu thập dữ liệu

Dữ liệu được lấy từ cổng dữ liệu mở của NASA [10] bằng module `cmapss.data.acquire`. Script tải gói `CMAPSSData.zip`, kiểm tra MD5 của gói, giải nén 12 file dữ liệu của bốn subset, rồi so

SHA-256 của từng file với giá trị đã ghi nhận. File đã có trong `data/raw` chỉ được giữ nguyên khi trùng nội dung; nếu khác, script dừng thay vì ghi đè. Lần chạy ngày 09/10/2026 cho thấy cả 12 file trùng với bản đang dùng, nghĩa là dữ liệu trong báo cáo đúng là bản gốc của nguồn. Nguồn, thời điểm tải, kích thước và hash của từng file được ghi vào `outputs/acquisition/manifest.json`, đóng vai trò hồ sơ của lớp Bronze.

4.2 Gom dữ liệu từ nhiều file

Mỗi subset có ba file. Hai file quan sát `train_FD001.txt` và `test_FD001.txt` có cùng 26 cột nhưng chứa hai nhóm động cơ khác nhau, nên chúng được **nối theo chiều dòng (union)**, không phải join. Trước khi nối, mỗi dòng được gắn thêm cột `split` để biết nó thuộc tập nào (Mã nguồn 1). Bảng kết quả của FD001 có $20.631 + 13.096 = 33.727$ dòng và 27 cột.

Mã nguồn 1: Đọc và gom hai file quan sát (rút gọn từ `pandas_pipeline.py` và `spark_pipeline.py`).

```
# Pandas: read each file, validate 26 fields, add the split column, then union rows
frames = [read_observations(raw_dir / f"{split}_FD001.txt", split=split)
          for split in ("train", "test")]
observations = pd.concat(frames, ignore_index=True)
observations = observations.sort_values(["split", "engine_id", "cycle"])

# PySpark: the same union on Spark DataFrames
observations = train.unionByName(test).orderBy("split", "engine_id", "cycle")
```

Cột `split` là cần thiết vì train engine 1 và test engine 1 là hai động cơ khác nhau nhưng có cùng `(engine_id, cycle)`. Sau khi nối, cặp `(engine_id, cycle)` bị trùng, còn bộ ba **`(split, engine_id, cycle)`** là duy nhất; đây là khóa của mọi bảng ở cấp cycle. File nhãn `RUL_FD001.txt` không được gom vào bảng này mà được giữ riêng đến bước đánh giá cuối. Bốn subset FD001–FD004 được xử lý ở bốn thư mục riêng; nếu muốn gộp thành một bảng thì phải thêm cột mã subset vào khóa.

4.3 Lưu trữ theo lớp

Bảng 8 liệt kê các bảng được lưu. Parquet lưu dữ liệu theo cột và giữ nguyên kiểu dữ liệu, phù hợp khi phân tích chỉ đọc một vài sensor. SQLite chứa thêm hai bảng nhỏ (`engine_summary`, `train_targets`) có index theo khóa, để truy vấn nhanh bằng SQL.

Bảng 8: Các bảng được lưu cho FD001.

Lớp	File	Một dòng là	Kích thước	Khóa
Bronze	`train_FD001.txt`, `test_FD001.txt`	1 cycle của 1 động cơ	33.727×26	không có header
Silver	`observations.parquet`	1 cycle của 1 động cơ	33.727×27	split, engine_id, cycle
Silver	`train_targets.parquet`	1 cycle train	20.631×4	split, engine_id, cycle
Gold	`engine_summary.parquet`	1 động cơ	200×5	split, engine_id
Gold	`etl.sqlite`	hai bảng trên	–	có index theo khóa
Gold	`rolling_features.parquet`	1 window 30 cycles	27.927×129	split, engine_id, cycle

Bảo vệ dữ liệu khi ghi. File gốc ở lớp Bronze không bao giờ bị ghi đè. ETL và rolling features ghi kết quả vào một thư mục tạm trước, kiểm tra xong mới thay bộ cũ; nếu bước thay thế lỗi, bộ cũ được khôi phục. File metadata (`run_info.json`) được ghi sau cùng và lưu hash của dữ liệu đầu vào, nên mỗi bộ kết quả luôn đi kèm đúng phiên bản dữ liệu đã tạo ra nó.

Kiểm thử. Bộ kiểm thử tự động gồm 453 kiểm thử đạt (một kiểm thử bỏ qua vì môi trường thiếu `torch`), lần chạy ngày 09/10/2026. Các kiểm thử bao gồm kiểm tra checksum khi thu thập, schema, khóa, ranh giới window, mapper và reducer của job MapReduce, công thức EWMA, quy tắc bất thường, lịch bảo trì và việc nhãn test chỉ được đọc sau khi đã ghi dự đoán.

4.4 Lưu trữ phân tán trên HDFS

Để thể hiện bước lưu trữ và xử lý phân tán của Hadoop, pipeline có thêm một cụm Hadoop 3.4.3 một node chạy trong Docker (`infra/hadoop/docker-compose.yml`): NameNode và DataNode cho HDFS, ResourceManager và NodeManager cho YARN, cùng một Spark 4.0.1 client. Lệnh `python -m cmapss.hdfs` thực hiện bốn bước:

1. đưa hai file raw FD001 lên thư mục `/cmapss/bronze/fd001` của HDFS;
2. chạy job MapReduce viết bằng Hadoop Streaming (`lifetime_job.py`) trên YARN để tính bảng Gold mỗi động cơ một dòng: số cycle, cycle đầu và cycle cuối;
3. chạy Spark đọc raw từ HDFS, ghi lớp Silver dạng Parquet phân vùng theo `split` vào `/cmapss/silver/observations`, rồi đọc lại Parquet đó để tính cùng bảng Gold;
4. tải hai bảng kết quả về và đối chiếu với bảng `engine_summary` do ETL Pandas tạo.

Mã nguồn 2: Mapper và reducer của job MapReduce (rút gọn từ `src/cmapss/hdfs/lifetime_job.py`).

```
# map: one raw line -> "split:engine<TAB>cycle"; the zero-padded id keeps Hadoop's key
    order
def map_lines(lines, source):
    split = split_from_path(source)             # train_FD001.txt -> "train"
    for line in lines:
```

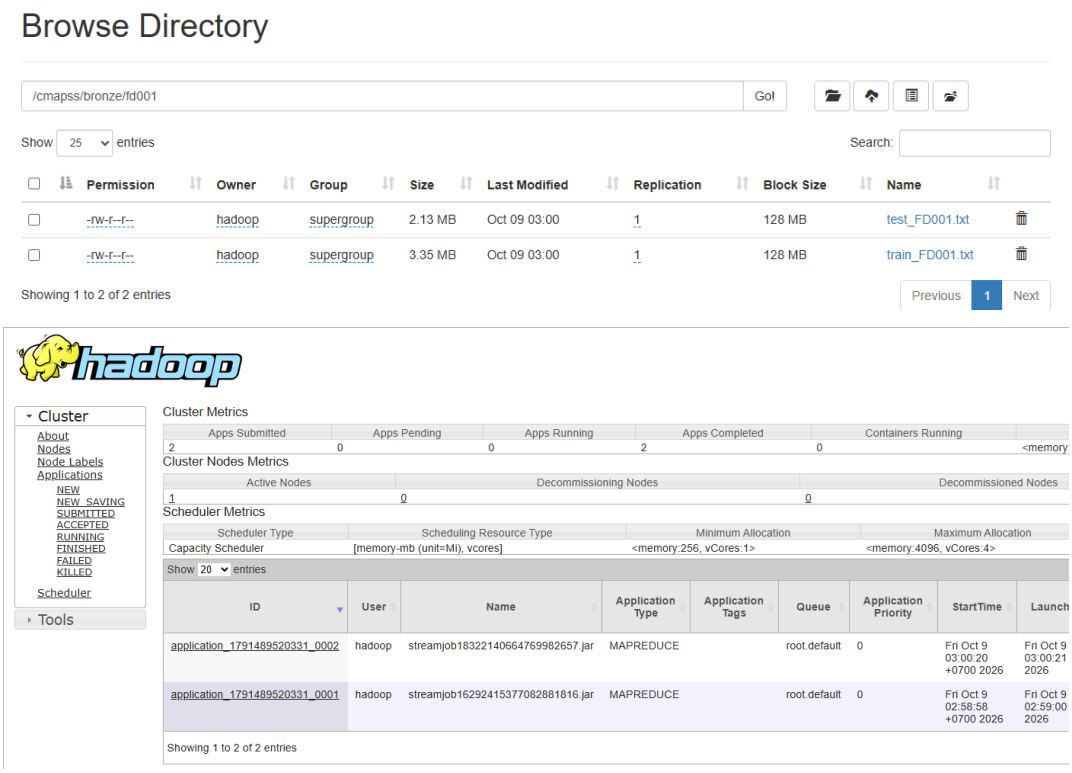
```
        fields = line.split()
        yield "%s:%04d\t%d" % (split, int(fields[0]), int(fields[1]))

# reduce: keys arrive sorted, so all cycles of one engine are consecutive
def reduce_lines(lines):
    current, cycles = None, []
    for line in lines:
        key, value = line.rstrip("\n").split("\t")
        if key != current and current is not None:
            yield summary(current, cycles)       # split, engine, n_cycles, min, max
            cycles = []
        current = key
        cycles.append(int(value))
    if current is not None:
        yield summary(current, cycles)
```

Bảng 9 tóm tắt lần chạy ngày 09/10/2026. Hadoop chia input thành ba phần (input split) và chạy ba map task cùng một reduce task. Cả MapReduce và Spark đều cho bảng 200 động cơ khớp tuyệt đối với Pandas. Hình 2 là giao diện web của NameNode và YARN trong lần chạy này.

Bảng 9: Kết quả bước HDFS trên cụm Hadoop một node (`outputs/hdfs/summary.json`).

Hạng mục	Kết quả
Bronze trên HDFS	2 file, 5,74 MB, replication 1
MapReduce trên YARN	3 map task, 1 reduce task; đọc 33.727 dòng, ghi 200 dòng; 22 giây
Spark trên HDFS	Silver 33.727 dòng, 0 giá trị thiếu, Parquet phân vùng theo `split`; 14 giây
Kiểm tra HDFS	`hdfs fsck /cmapss`: 6 block, 0 block thiếu bản sao, trạng thái HEALTHY
Đối chiếu với Pandas	MapReduce và Spark đều khớp 200/200 động cơ, 0 ô lệch



Hình 2: Trên: hai file raw FD001 trên HDFS, giao diện NameNode (replication 1, block 128 MB). Dưới: các job MapReduce đã hoàn thành trên YARN.

Cụm chỉ có một node nên bước này không đo lợi ích hiệu năng của xử lý phân tán. Mục đích là chứng minh cùng dữ liệu và cùng phép tính chạy được trên HDFS, MapReduce và Spark, cho kết quả giống hệt Pandas. Với file lớn hơn 128 MB, HDFS sẽ chia file thành nhiều block trên nhiều DataNode và YARN chạy nhiều map task song song trên các block đó.

4.5 Truy vấn dữ liệu

Dữ liệu được lấy ra theo ba cách, tùy loại bảng.

SQL trên SQLite. Các bảng tổng hợp được truy vấn trực tiếp bằng SQL. Truy vấn trong Mã nguồn 3 join bảng tổng hợp với bảng nhãn để lấy số cycle và RUL của train engine 1 tại ba thời điểm.

Mã nguồn 3: Truy vấn SQLite `outputs/etl/gold/etl.sqlite`.

```sql
SELECT s.engine_id, s.n_cycles, t.cycle, t.rul
FROM engine_summary AS s
JOIN train_targets  AS t
  ON t.split = s.split AND t.engine_id = s.engine_id
WHERE s.split = 'train' AND s.engine_id = 1 AND t.cycle IN (1, 100, 192);
-- engine_id | n_cycles | cycle | rul
--     1     |   192    |   1   | 191
```

```
--     1     |   192    |  100  |  92
--     1     |   192    |  192  |   0
```

Lọc và chọn cột trên Parquet bằng Pandas. Dữ liệu sensor chi tiết được đọc từ Parquet, rồi lọc dòng (selection) và chọn cột (projection). Ví dụ, lấy `sensor_11` của train engine 1 trong cycles 71–100 trả về 30 dòng với giá trị trung bình 47,395.

Spark SQL trên Parquet. Cùng file Parquet có thể được đăng ký thành bảng tạm và truy vấn bằng Spark SQL, gồm cả `GROUP BY`, `JOIN` và window function. Mã nguồn 4 dùng window function để tính trung bình 30 cycles gần nhất của `sensor_11`, chính là phép tính của đặc trưng `sensor_11__mean`. Kết quả tại cycle 100 (47,395) trùng với kết quả Pandas. Phép lọc cycle phải nằm ở truy vấn ngoài: nếu đặt cùng cấp với window function, window chỉ nhìn thấy các dòng đã lọc và cho kết quả sai. Spark chạy ở chế độ local trên một máy; đây là minh họa cách truy vấn, không phải phép đo hiệu năng phân tán.

Mã nguồn 4: Spark SQL trên `observations.parquet`: trung bình 30 cycles gần nhất.

```sql
SELECT engine_id, cycle, n_rows_in_window, ROUND(mean30, 3) AS mean30
FROM (
  SELECT engine_id, cycle,
         COUNT(*)       OVER w AS n_rows_in_window,
         AVG(sensor_11) OVER w AS mean30
  FROM observations
  WHERE split = 'train' AND engine_id = 1
  WINDOW w AS (PARTITION BY split, engine_id ORDER BY cycle
               ROWS BETWEEN 29 PRECEDING AND CURRENT ROW)
)
WHERE cycle IN (29, 30, 100);
-- cycle 29:  29 rows, mean30 = 47.279   (window not full: no feature row)
-- cycle 30:  30 rows, mean30 = 47.283
-- cycle 100: 30 rows, mean30 = 47.395
```

4.6 Các bước biến đổi

Biến đổi ở đây là bước T trong ETL: đổi kiểu, gom dòng, tổng hợp, tạo nhãn và tạo đặc trưng. Bảng 10 liệt kê sáu bước theo thứ tự, kèm kích thước bảng sau mỗi bước.

Bảng 10: Sáu bước biến đổi dữ liệu và kích thước sau mỗi bước (FD001).

#	Đầu vào → đầu ra	Phép biến đổi	Kết quả
1	2 file text → 2 bảng có kiểu	Tách 26 trường, gán tên cột và kiểu; kiểm tra khóa và cycle liên tục; thêm `split`	20.631 và 13.096 dòng
2	2 bảng → observations	UNION theo dòng, sắp theo khóa	33.727×27
3	observations train → train_targets	GROUP BY động cơ, lấy max(cycle); rul = max(cycle) − cycle	20.631×4
4	observations → engine_summary	GROUP BY `(split, engine_id)`: đếm cycle, lấy cycle đầu và cuối	200×5
5	observations → rolling_features	Với mỗi động cơ: window 30 cycles, 6 thống kê × 21 sensor	27.927×129
6	đặc trưng train + nhãn → X, y	JOIN theo đủ 3 khóa, bắt buộc một-một	X: 17.731×126

Mã nguồn 5 trích phần code của bước 3, 5 và 6. Bước 3 được viết bằng cả Pandas và PySpark với cùng logic.

Mã nguồn 5: Tạo nhãn, tạo đặc trưng và join (trích từ `targets.py`, `spark_pipeline.py`, `rolling.py`, `modeling/pipeline.py`).

```
# Step 3 - Pandas: RUL = last cycle of the engine - current cycle
last_cycle = observations.groupby("engine_id")["cycle"].transform("max")
targets["rul"] = last_cycle - observations["cycle"]

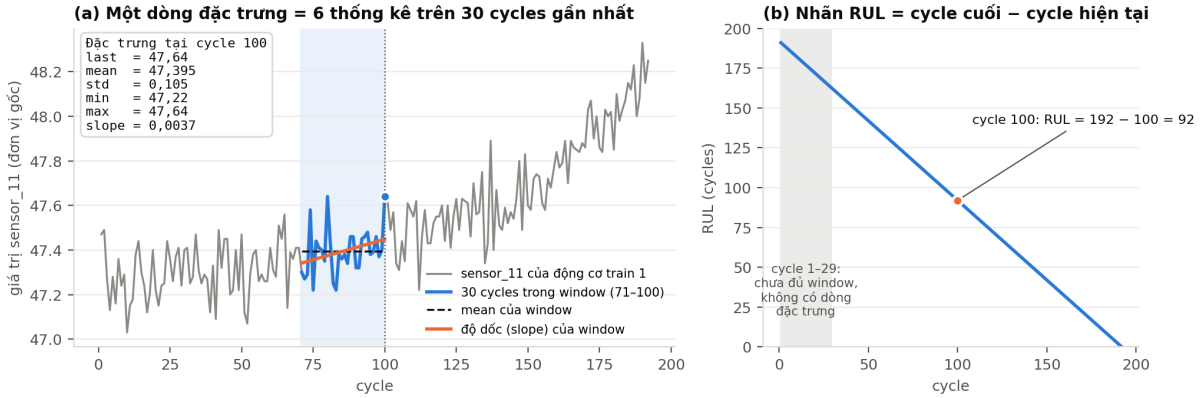
# Step 3 - PySpark: the same label with groupBy + join
max_cycles = train.groupBy("engine_id").agg(F.max("cycle").alias("max_cycle"))
targets = (train.join(max_cycles, on="engine_id")
                .withColumn("rul", F.col("max_cycle") - F.col("cycle")))

# Step 5 - for every (split, engine_id) group, sorted by cycle
rolling = values.rolling(30, min_periods=30)
feature[f"{sensor}__last"]  = values
feature[f"{sensor}__mean"]  = rolling.mean()
feature[f"{sensor}__std"]   = rolling.std(ddof=1)
feature[f"{sensor}__min"]   = rolling.min()
feature[f"{sensor}__max"]   = rolling.max()
feature[f"{sensor}__slope"] = rolling.apply(_slope, raw=True)

# Step 6 - one-to-one join on the full key
dataset = train_features.merge(train_targets, on=["split", "engine_id", "cycle"],
                               how="left", validate="one_to_one")
```

Hình 3 minh họa bước 5 và bước 3 trên train engine 1. Tại cycle 100, window gồm cycles 71–100; sáu thống kê của `sensor_11` là giá trị cuối 47,64, trung bình 47,395, độ lệch chuẩn 0,105, min 47,22, max 47,64 và độ dốc 0,0037 mỗi cycle. Nhãn của dòng này là $192 - 100 = 92$. Giá

trị ở cycle 101 trở đi không tham gia tính toán. Vì 29 cycle đầu chưa đủ window, một động cơ dài T cycles tạo ra $T - 29$ dòng đặc trưng; với FD001 là $20.631 - 100 \times 29 = 17.731$ dòng train và $13.096 - 100 \times 29 = 10.196$ dòng test.



Hình 3: Minh họa bước tạo đặc trưng và nhãn trên train engine 1. (a) Sáu thống kê của `sensor_11` trên window cycles 71–100 tạo thành phần đặc trưng của dòng cycle 100. (b) Nhãn RUL giảm tuyến tính về 0 tại cycle hỏng.

ETL chỉ chuẩn hóa schema và kiểu dữ liệu, không scale giá trị sensor. Việc scale chỉ diễn ra bên trong mô hình Ridge (StandardScaler fit trong từng fold) và khi tạo Health Index; HGB dùng đặc trưng ở thang đo gốc.

Song song với sáu bước trên, nhánh phân tích tín hiệu làm mượt 14 sensor có xu hướng bằng EWMA, rồi tạo Health Index cho mọi cycle (Mục 5.2); nhánh này không cung cấp cột nào cho mô hình RUL.

Để kiểm tra tính đúng của ETL, PySpark thực hiện lại bước 1–5 và bộ đối chiếu so sánh schema, khóa, số dòng và từng giá trị với kết quả Pandas. Benchmark đọc/ghi, rolling features và đối chiếu Pandas–PySpark được trình bày ở Mục 6.1; dữ liệu nhân bản dùng cho benchmark không được dùng để huấn luyện.

4.7 Cột được dùng cho mô hình

Bảng 11 cho biết mỗi nhóm cột được dùng vào việc gì. Mô hình chỉ nhận 126 cột đặc trưng, đặt tên theo mẫu `<sensor>__<thống kê>`, ví dụ `sensor_11__mean`. Ba cột khóa chỉ dùng để join và chia fold theo động cơ. Ba operational settings không nằm trong tập đặc trưng: FD001 chỉ có một điều kiện vận hành nên settings gần như không đổi; với FD002 và FD004, lựa chọn này trở thành một giới hạn (Mục 7.3).

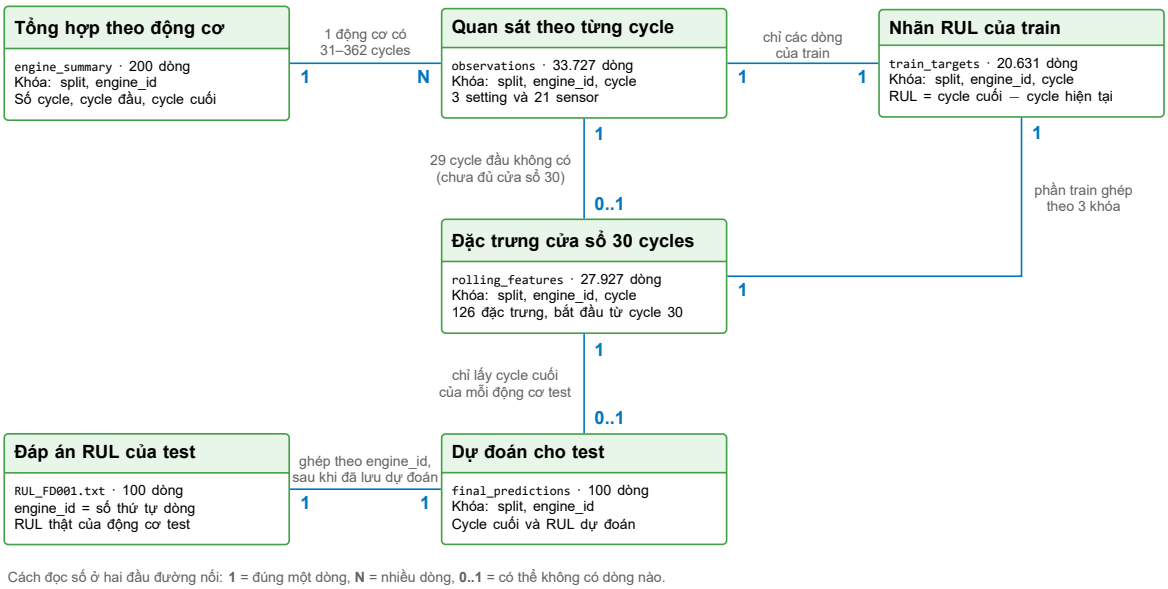
Bảng 11: Vai trò của các nhóm cột.

Nhóm cột	Số cột	Vai trò	Trong X?
`split`, `engine_id`, `cycle`	3	Khóa: join, chia fold theo động cơ	Không
`setting_1`–`setting_3`	3	Điều kiện vận hành	Không
`sensor_1`–`sensor_21`	21	Tín hiệu gốc, dùng qua đặc trưng window; bản EWMA dùng cho Health Index	Không
`sensor_k__{last, mean, std, min, max, slope}`	126	Đặc trưng window 30 cycles	Có
`rul`	1	Nhãn (biến mục tiêu y)	Là y

Báo cáo không thực hiện chọn lọc đặc trưng (feature selection): toàn bộ 126 cột được giữ, kể cả các cột sinh ra từ sensor hằng. Cách làm này tránh dùng kết quả EDA hoặc p-value trên toàn bộ train để chọn cột, vốn là một dạng data leakage đối với cross-validation.

4.8 Quan hệ giữa các bảng

Hình 4 và Bảng 12 mô tả quan hệ giữa các bảng. Quan hệ duy nhất có bậc nhiều là giữa động cơ và quan sát (**1:N**): một động cơ có từ 31 đến 362 cycles. Các quan hệ còn lại là **1:1** hoặc **1:0..1**. Mỗi quan hệ được kiểm tra trực tiếp trên dữ liệu trong notebook walkthrough; kết quả ghi ở cột cuối của bảng.



Hình 4: Quan hệ giữa các bảng trong một subset. Số ở hai đầu mỗi đường là bậc quan hệ; "0..1" nghĩa là một dòng bên kia có thể có hoặc không có dòng tương ứng.

Bảng 12: Quan hệ giữa các bảng và kết quả kiểm tra trên FD001.

Hai bảng	Quan hệ	Kiểm tra trên dữ liệu
engine_summary – observations	1 : N	Mỗi động cơ có 31–362 cycles
observations (train) – train_targets	1 : 1	20.631 / 20.631 dòng khớp khóa
observations – rolling_features	1 : 0..1	27.927 dòng có đặc trưng; 5.800 dòng (29 cycle đầu $\times$ 200 động cơ) không có
rolling_features (train) – train_targets	1 : 1	17.731 dòng sau join, đúng bằng số dòng đặc trưng train
final_predictions – test_labels	1 : 1	100 dòng, 100 động cơ test khác nhau

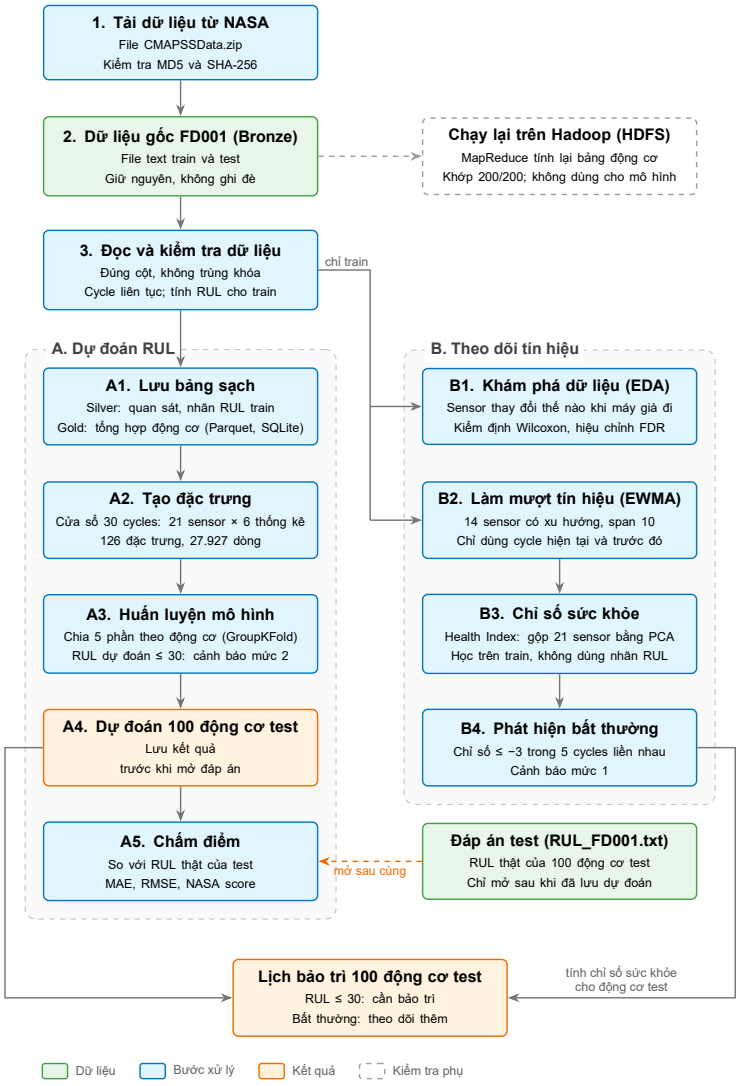
Không có quan hệ nhiều-nhiều. Một window dùng 30 quan sát, và hai window liên tiếp dùng chung 29 quan sát. Đây là phụ thuộc theo thời gian khi tính toán, không phải một phép join nhiều-nhiều giữa hai bảng: mỗi dòng đặc trưng vẫn gắn với đúng một cycle cuối của window.

Việc join đúng khóa quan trọng vì một lỗi nhỏ có thể làm nhân số dòng. Với train engine 1 (163 dòng đặc trưng, 192 dòng nhãn), join chỉ theo `engine_id` ghép mọi dòng đặc trưng với mọi dòng nhãn, tạo ra $163 \times 192 = 31.296$ dòng; join theo đủ ba khóa trả về đúng 163 dòng. SQLite chưa khai báo ràng buộc khóa ngoại (foreign key); thay vào đó, code kiểm tra quan hệ một-một bằng tham số `validate="one_to_one"` và dừng nếu vi phạm.

5 Phương pháp

5.1 Tổng quan pipeline

Hình 5 tách nhánh A (dự đoán RUL) và nhánh B (theo dõi tín hiệu). Rolling features đọc trực tiếp Silver observations; Gold `engine_summary` là bảng tổng hợp riêng. Nhánh tín hiệu phân tích train bằng EDA, EWMA, Health Index và quy tắc bất thường; nhánh RUL dùng sensor gốc qua rolling statistics. Lịch bảo trì kết hợp hai đầu ra: RUL dự đoán và HI của test sau EWMA, transform bằng scaler/PCA fit trên train (Mục 5.7). Nhãn test được đọc sau khi ghi dự đoán hoặc lịch. HDFS/MapReduce/Spark đối chiếu bảng tổng hợp trên một node Docker, độc lập với luồng huấn luyện RUL bằng Pandas/scikit-learn.



Hình 5: Luồng xử lý dữ liệu. Bước 1–3 chuẩn bị dữ liệu; nhánh A dự đoán RUL, nhánh B theo dõi tín hiệu và chỉ học trên train. Ô nét đứt (chạy lại trên Hadoop) chỉ kiểm tra lại bảng tổng hợp động cơ, không đi vào mô hình. Đáp án test chỉ được mở sau khi đã lưu dự đoán; lịch bảo trì kết hợp hai nhánh. Số liệu trong hình là của FD001.

Bảng 13 liệt kê công cụ ở từng bước. Pandas là implementation chính; PySpark thực hiện

lại phần ETL và rolling features để đối chiếu kết quả. Cách gom, lưu trữ, truy vấn và biến đổi dữ liệu đã được trình bày ở Mục 4; các mục dưới đây mô tả phần phân tích.

Bảng 13: Công cụ và đầu ra của từng bước.

Bước	Công cụ	Đầu ra
Thu thập	Python (urllib), MD5, SHA-256	File raw và manifest nguồn
Đọc và kiểm tra	Python, Pandas	Bảng quan sát đã kiểm tra, nhãn RUL train
ETL	Pandas, PySpark, Parquet, SQLite	Lớp Silver/Gold, báo cáo parity
Lưu trữ phân tán	Docker, Hadoop 3.4.3 (HDFS, YARN, MapReduce), Spark 4.0.1	Bronze và Silver trên HDFS, bảng Gold từ MapReduce và Spark
Khử nhiễu, bất thường	Pandas, scikit-learn	Tín hiệu EWMA, Health Index, cycle phát hiện bất thường
Tạo đặc trưng	Pandas, PySpark	126 đặc trưng window 30 cycles
Mô hình	scikit-learn	GroupKFold, Ridge, HistGradientBoosting
Đánh giá, cảnh báo	scikit-learn, SciPy	MAE, RMSE, NASA score, bootstrap, lead time
Lịch bảo trì	Pandas	Cycle cần bảo trì và trạng thái cho 100 động cơ test
Trực quan hóa	Matplotlib, Seaborn, Streamlit	Hình trong báo cáo, dashboard chỉ đọc kết quả

5.2 Khử nhiễu và Health Index

Mỗi sensor được làm mượt riêng trong từng động cơ bằng exponentially weighted moving average (EWMA, trung bình trượt có trọng số mũ):

$$s_{i,t} = \alpha\, x_{i,t} + (1-\alpha)\, s_{i,t-1}, \qquad s_{i,1} = x_{i,1}, \qquad \alpha = \frac{2}{\text{span}+1}. \tag{7}$$

Với span 10 thì $\alpha \approx 0{,}18$: giá trị mới chiếm 18%, phần còn lại là lịch sử đã làm mượt. EWMA chỉ dùng cycle hiện tại và quá khứ, nên không rò rỉ thông tin tương lai. EWMA được áp dụng cho 14 sensor có xu hướng (2, 3, 4, 7, 8, 9, 11, 12, 13, 14, 15, 17, 20, 21); bảy sensor còn lại hằng hoặc gần hằng. Span 10 được cố định trước khi chạy và không được tinh chỉnh theo kết quả. Mức khử nhiễu được đo bằng độ lệch chuẩn của sai phân $x_{i,t} - x_{i,t-1}$ trước và sau làm mượt: xu hướng suy giảm thay đổi chậm nên đóng góp ít vào sai phân, còn dao động giữa hai cycle liền kề đóng góp phần lớn.

Health Index (HI, chỉ số sức khỏe) gộp 21 sensor thành một chuỗi: 14 sensor có xu hướng đã được làm mượt bằng EWMA, 7 sensor gần như không đổi giữ nguyên. Các sensor được chuẩn hóa bằng StandardScaler (trừ trung bình, chia độ lệch chuẩn), chiếu lên thành phần chính thứ nhất của PCA, đổi dấu để HI giảm theo cycle, rồi chuẩn hóa về trung bình 0, độ lệch chuẩn

1. Bước này không dùng nhãn RUL.

5.3 Phát hiện dấu hiệu bất thường

Mỗi động cơ được so sánh với chính nó ở giai đoạn vận hành ổn định đầu vòng đời. Gọi μ_i và σ_i là trung bình và độ lệch chuẩn HI trong 30 cycles đầu của động cơ i. Tại cycle $t > 30$,

$$z_{i,t} = \frac{h_{i,t} - \mu_i}{\sigma_i}. \tag{8}$$

Động cơ được báo bất thường tại cycle đầu tiên mà $z_{i,t} \leq -3$ đã giữ liên tiếp 5 cycles. Ngưỡng 3 độ lệch chuẩn và yêu cầu 5 cycles liên tiếp được cố định trước khi chạy; yêu cầu liên tiếp giúp một gai đơn lẻ không gây báo động. Ví dụ, nếu $\mu_i = 0{,}94$ và $\sigma_i = 0{,}07$ thì ngưỡng là $0{,}94 - 3 \times 0{,}07 = 0{,}73$. Kết quả được tóm tắt bằng số cycle từ lúc phát hiện đến khi hỏng. Vì mọi động cơ train đều đi đến hỏng và dữ liệu không ghi thời điểm bắt đầu suy giảm, phương pháp này không đo được trực tiếp tỷ lệ dương tính giả. Thay vào đó, quy tắc được chạy lại trên lưới ngưỡng $z \in \{2, 3, 4\}$ và số cycle liên tiếp $k \in \{3, 5, 10\}$; với mỗi cấu hình, báo cáo ghi số động cơ phát hiện được, lead time và số động cơ bị báo trong 10 cycles ngay sau đoạn tham chiếu, dùng làm chỉ báo gián tiếp cho độ nhạy quá mức (kết quả ở Bảng 18).

5.4 Kiểm định xu hướng đầu/cuối vòng đời

Với sensor s của động cơ i, báo cáo tính trung bình của 30 cycles đầu và 30 cycles cuối, rồi lấy chênh lệch:

$$E_{i,s} = \frac{1}{30}\sum_{t=1}^{30} x_{i,t,s}, \qquad L_{i,s} = \frac{1}{30}\sum_{t=T_i-29}^{T_i} x_{i,t,s}, \qquad \Delta_{i,s} = L_{i,s} - E_{i,s}. \tag{9}$$

Động cơ ngắn nhất dài 128 cycles, nên hai đoạn không bao giờ chồng nhau. Khoảng tin cậy 95% của trung bình chênh lệch dùng bootstrap percentile 10.000 lần lấy mẫu lại động cơ, seed 42. Kiểm định Wilcoxon hai phía dùng 100.000 hoán vị, sau đó hiệu chỉnh Benjamini–Hochberg với $q = 0{,}05$ trên cả 21 sensor. Kết quả mô tả xu hướng trên train và không được dùng để chọn đặc trưng cho mô hình.

5.5 Huấn luyện và đánh giá mô hình RUL

Ba mô hình được so sánh:

- **Median baseline**: dự đoán một hằng số bằng trung vị có trọng số của RUL trong fold huấn luyện.
- **Ridge**: hồi quy tuyến tính với phạt L2, $\alpha = 1$; đặc trưng được chuẩn hóa bằng `StandardScaler` fit trong từng fold huấn luyện.
- **HistGradientBoosting (HGB)**: tổ hợp cây hồi quy với 200 vòng boosting, learning rate 0,05, tối đa 31 lá mỗi cây, tối thiểu 20 mẫu mỗi lá, tắt early stopping, `random_state=42`; dùng đặc

trưng ở thang đo gốc vì cây quyết định không phụ thuộc thang đo.

Mô hình được chọn bằng GroupKFold 5 fold theo `engine_id`: mỗi fold có 20 động cơ, và mọi window của một động cơ nằm cùng một fold. Chia ngẫu nhiên theo dòng sẽ đưa các window gần như trùng nhau của cùng động cơ vào cả train và validation, làm sai số validation thấp hơn thực tế. Vì động cơ sống lâu có nhiều window hơn, trọng số mẫu được chuẩn hóa để tổng trọng số của mỗi động cơ bằng 1.

Tiêu chí chọn là MAE trung bình theo động cơ; RMSE và NASA score được ghi kèm. Mô hình có MAE thấp nhất được huấn luyện lại trên toàn bộ 17.731 window train và dự đoán RUL tại cycle cuối của mỗi động cơ test. Chỉ sau khi 100 dự đoán này đã được ghi ra file, pipeline mới đọc `RUL_FD001.txt` để tính metric cuối. Khoảng tin cậy 95% của metric test được ước lượng bằng bootstrap 10.000 lần, lấy mẫu lại động cơ có hoàn lại, seed 42.

5.6 Cảnh báo sớm

Pipeline chuyển dự đoán OOF trên train thành cảnh báo: động cơ được cảnh báo tại cycle đầu tiên mà RUL dự đoán không vượt quá 30 cycles. Ngưỡng 30 được cố định trước, không tối ưu trên dữ liệu; ngưỡng 40 chỉ dùng để phân tích độ nhạy. Tập train được dùng vì mỗi động cơ có toàn bộ quỹ đạo đến khi hỏng, trong khi test chỉ có một điểm cắt.

Với động cơ i, đặt a_i là cycle cảnh báo đầu tiên ($\min\{t : \hat{y}_{i,t} \le 30\}$) và r_i là cycle đầu tiên RUL thật chạm ngưỡng ($\min\{t : y_{i,t} \le 30\}$). Hai đại lượng được ghi lại là:

- **Lead time** $\ell_i = T_i - a_i$: số cycle từ cảnh báo đến khi hỏng.
- **Độ lệch so với ngưỡng** $\delta_i = a_i - r_i$: $\delta_i < 0$ là cảnh báo trước ngưỡng, $\delta_i = 0$ là đúng ngưỡng, $\delta_i > 0$ là cảnh báo trễ.

Động cơ không bao giờ có dự đoán dưới ngưỡng được ghi là bỏ lỡ. Vì mọi động cơ train đều đi đến hỏng, không tồn tại động cơ "âm tính", nên tỷ lệ báo động giả ở cấp động cơ không xác định. Cùng tập OOF này đã được dùng để chọn mô hình, nên kết quả cảnh báo là đánh giá nội bộ, có phần lạc quan.

5.7 Báo cáo dự đoán thời điểm cần bảo trì

Bước cuối ghép hai mức cảnh báo thành một bảng cho 100 động cơ test, tại cycle cuối được quan sát t_i của mỗi động cơ. Với RUL dự đoán $\hat{y}_i$ và ngưỡng 30:

$$c_i = \max\big(\lfloor \hat{y}_i - 30 \rfloor,\, 0\big), \qquad m_i = t_i + c_i, \tag{10}$$

trong đó c_i là số cycle còn lại đến lúc cần bảo trì và m_i là cycle dự kiến cần bảo trì. Làm tròn xuống giúp lịch nghiêng về phía sớm hơn. Ví dụ, động cơ 90 ở cycle 146 có $\hat{y} = 31{,}2$, nên $c = 1$ và $m = 147$.

Trạng thái được xét theo thứ tự ưu tiên cố định trước khi chạy: *cần bảo trì* nếu $\hat{y}_i \le 30$; *theo dõi tăng cường* nếu Health Index của động cơ đã báo bất thường theo quy tắc ở Mục 5.3; còn lại là *bình thường*. Health Index của động cơ test dùng đúng scaler, PCA và hệ số chuẩn hóa đã fit trên train, không fit lại. Động cơ có ít hơn 35 cycles (30 cycles tham chiếu và 5 cycles

liên tiếp) được ghi là chưa đủ dữ liệu để đánh giá bất thường. Bảng được ghi ra file trước; chỉ sau đó `RUL_FD001.txt` mới được đọc để kiểm tra.

5.8 Thử nghiệm mở rộng

Ba thử nghiệm phụ dùng cùng nguyên tắc chia theo động cơ và đọc nhãn test muộn:

- **Cải thiện mô hình**: nested GroupKFold (5 fold ngoài, 3 fold trong) chọn window, siêu tham số và trọng số chỉ bằng fold trong; tiêu chí chấp nhận được chốt trước khi chạy [16, 14].
- **Chuyển miền**: mô hình chọn và huấn luyện trên FD001 được áp dụng nguyên trạng lên test FD002–FD004, so với mô hình huấn luyện nội bộ từng subset. Window 10 cycles được dùng vì động cơ test ngắn nhất của FD004 chỉ có 19 cycles, nên kết quả không so trực tiếp với window 30.
- **ARIMA**: trên 10 động cơ train chọn trước, Health Index được chia 70% đầu để fit ARIMA (bậc chọn bằng AIC) và 30% cuối để đánh giá dự báo, so với hai baseline `last_value` và `drift`.

6 Kết quả

Kết quả được trình bày theo đúng thứ tự của pipeline: dữ liệu được thu thập, kiểm tra và lưu trữ (Mục 6.1); tín hiệu được khử nhiễu và tổng hợp theo window (Mục 6.2); xu hướng suy giảm và dấu hiệu bất thường được phát hiện (Mục 6.3); RUL được dự đoán (Mục 6.4); chuyển thành cảnh báo sớm (Mục 6.5); và cuối cùng thành báo cáo dự đoán thời điểm cần bảo trì cho từng động cơ (Mục 6.6). Các thử nghiệm ngoài luồng chính được gom ở Mục 6.7. Trừ khi ghi khác, mọi số liệu thuộc FD001.

6.1 Pipeline dữ liệu và chất lượng dữ liệu

Cả ba bảng nguồn (quan sát train, nhãn train, quan sát test) đều vượt qua mọi kiểm tra về schema, khóa, kiểu số và tính liên tục của cycle. Tập train có 20.631 dòng và tập test có 13.096 dòng, mỗi tập 100 động cơ; 100 nhãn test hợp lệ về cấu trúc. Không có giá trị thiếu, nên pipeline không cần điền hay xóa dòng.

ETL tạo lớp Silver gồm 33.727 quan sát train/test và 20.631 nhãn train, cùng lớp Gold gồm 200 dòng engine summary trong Parquet và SQLite. Mọi bảng được đọc lại sau khi ghi để kiểm tra. Trên cả ba bảng, kết quả PySpark khớp hoàn toàn với Pandas về schema, khóa, số dòng và giá trị; sai khác tuyệt đối lớn nhất bằng 0. Rolling features 30 cycles tính bằng Spark cũng khớp toàn bộ 27.927 dòng của Pandas với tolerance 10^{-7}. Trên cụm Hadoop một node, MapReduce và Spark đọc dữ liệu từ HDFS cũng cho bảng engine summary khớp tuyệt đối với Pandas (200/200 động cơ, Mục 4.4).

Bảng 14 đo toàn bộ pipeline (đọc Parquet, tổng hợp, rolling features, ghi) khi nhân bản FD001 lên 10 lần. Thời gian Pandas tăng gần tuyến tính, từ 10,54 lên 111,68 giây; Spark tăng từ 14,60 lên 32,68 giây. Ở scale $1\times$ Pandas nhanh hơn vì Spark tốn chi phí khởi động và điều phối; ở scale $10\times$ Spark nhanh hơn khoảng 3,4 lần nhờ chia việc cho bốn luồng.

Bảng 14: Benchmark local, trung vị ba lượt; thời gian giây, RSS gồm Python và JVM. Cột Pipeline không gồm thời gian khởi động Spark (5,5 giây).

Engine	Scale	Đọc	Feature	Ghi	Pipeline	RSS (GiB)
pandas	1	0,05	10,30	0,15	10,54	0,25
pandas	10	0,09	110,39	0,97	111,68	1,73
spark	1	4,88	7,03	1,43	14,60	1,48
spark	10	5,28	22,12	4,08	32,68	3,05

Phép đo dùng Spark `local[4]` trên một máy Windows, heap 4 GiB, ghi qua driver; dữ liệu nhân bản chỉ phục vụ đo hiệu năng, không dùng để huấn luyện. Kết quả cho thấy lý do giữ implementation Spark khi dữ liệu tăng, nhưng chưa phải bằng chứng trên cluster nhiều máy.

Tuổi thọ của 100 động cơ train dao động từ 128 đến 362 cycles (Bảng 15). Phân phối lệch phải: trung bình 206,31 cycles lớn hơn trung vị 199 cycles do một số động cơ sống rất lâu.

Bảng 15: Thống kê tuổi thọ của 100 động cơ train FD001, đơn vị cycle.

Thống kê	Giá trị	Thống kê	Giá trị
Trung bình	206,31	Độ lệch chuẩn	46,34
Nhỏ nhất	128	Lớn nhất	362
Tứ phân vị 25%	177	Trung vị	199
Tứ phân vị 75%	229,25	Số động cơ	100

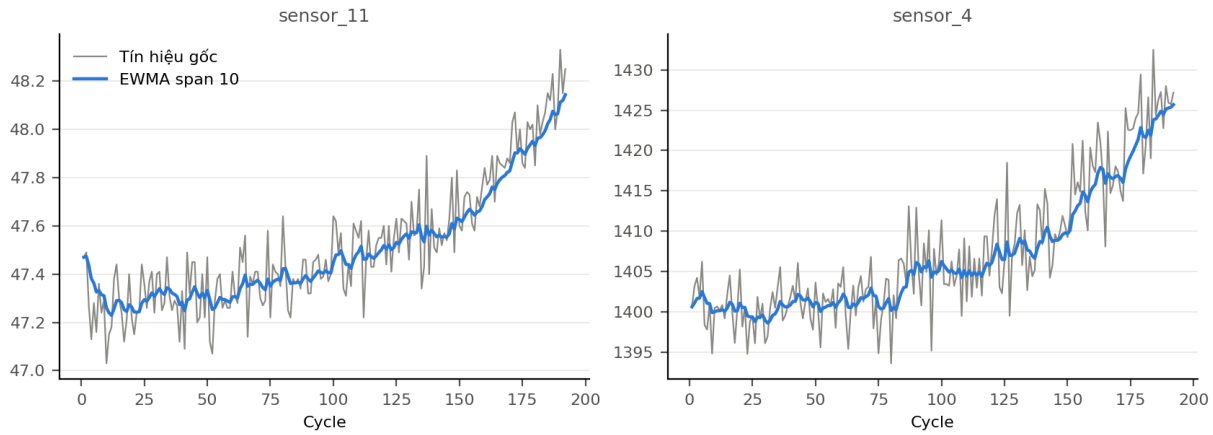
6.2 Xử lý nhiễu và tổng hợp theo cửa sổ thời gian

Tín hiệu sensor gốc dao động mạnh giữa hai cycle liền kề, che khuất xu hướng suy giảm chậm. EWMA span 10 (Mục 5.2) giảm độ lệch chuẩn của sai phân bậc một từ 83,6% đến 86,4% trên 14 sensor có thông tin, trung vị 86,3% (Bảng 16). Ví dụ, với `sensor_4`, độ lệch chuẩn của bước nhảy giữa hai cycle giảm từ 5,70 xuống 0,78. Mức giảm gần như bằng nhau ở mọi sensor; điều này phù hợp với giả định dao động ngắn hạn gần độc lập giữa các cycle, nhưng không chứng minh đó là nhiễu đo thuần túy vì dữ liệu không có tín hiệu sạch để đối chiếu.

Bảng 16: Độ lệch chuẩn sai phân bậc một trong từng động cơ, trước và sau EWMA span 10; một số sensor tiêu biểu trong 14 sensor có thông tin.

Sensor	Tín hiệu gốc	Sau EWMA	Mức giảm
`sensor_4`	5,6994	0,7829	86,3%
`sensor_9`	5,9037	0,9241	84,3%
`sensor_11`	0,1436	0,0198	86,2%
`sensor_12`	0,4245	0,0588	86,2%
`sensor_14`	4,4532	0,7285	83,6%

Hình 6 cho thấy hiệu quả trên động cơ train 1. Đường xám là tín hiệu gốc; đường xanh sau EWMA vẫn giữ đúng hướng tăng của `sensor_11` và `sensor_4`, đặc biệt là đoạn tăng nhanh sau cycle 150, nhưng bỏ đi phần lớn các gai ngắn hạn.

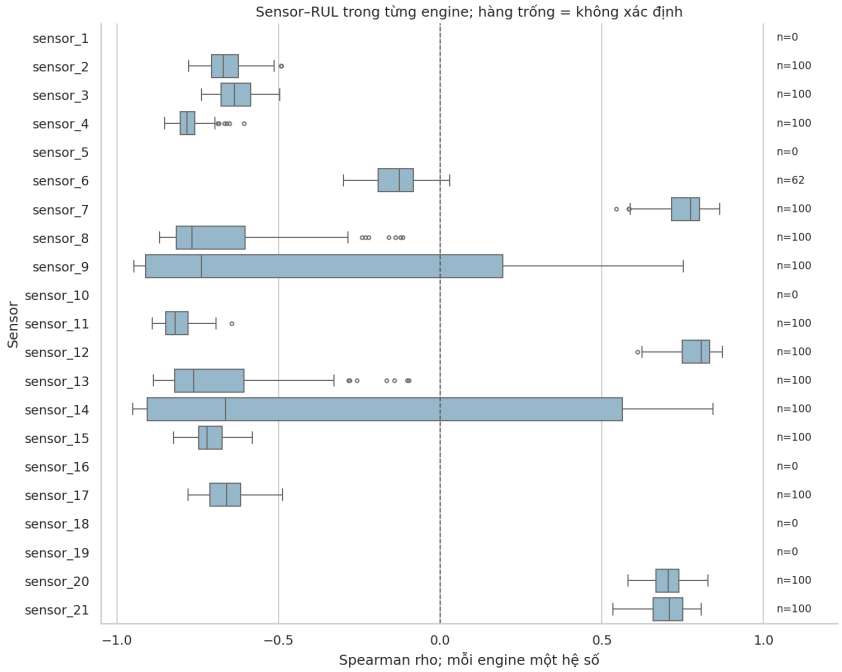


Hình 6: Tín hiệu `sensor_11` và `sensor_4` của động cơ train 1 trước (xám) và sau (xanh) khi làm mượt bằng EWMA span 10. EWMA chỉ dùng giá trị hiện tại và quá khứ.

Bước tổng hợp theo cửa sổ thời gian dùng window trượt 30 cycles (Mục 4.6): mỗi dòng đặc trưng gồm sáu thống kê (giá trị cuối, trung bình, độ lệch chuẩn, min, max, độ dốc) của 21 sensor, tổng cộng 126 đặc trưng. Trung bình và độ dốc trên 30 cycles cũng làm dịu dao động, nên mô hình RUL dùng trực tiếp tín hiệu gốc qua window thay vì tín hiệu EWMA; tín hiệu EWMA được dùng cho Health Index và phát hiện bất thường ở mục kế tiếp.

6.3 Xu hướng suy giảm và dấu hiệu bất thường

Tương quan Spearman giữa từng sensor và RUL được tính riêng trong mỗi động cơ, rồi tóm tắt qua 100 động cơ (Hình 7). Sáu sensor hằng (1, 5, 10, 16, 18, 19) không có tương quan xác định. Nhiều sensor có tương quan mạnh và nhất quán, ví dụ `sensor_11` (trung vị $-0{,}8190$) và `sensor_12` (trung vị $0{,}8075$): `sensor_11` tăng và `sensor_12` giảm khi động cơ tiến gần đến hỏng.



Hình 7: Phân phối hệ số tương quan Spearman sensor–RUL qua các động cơ train.

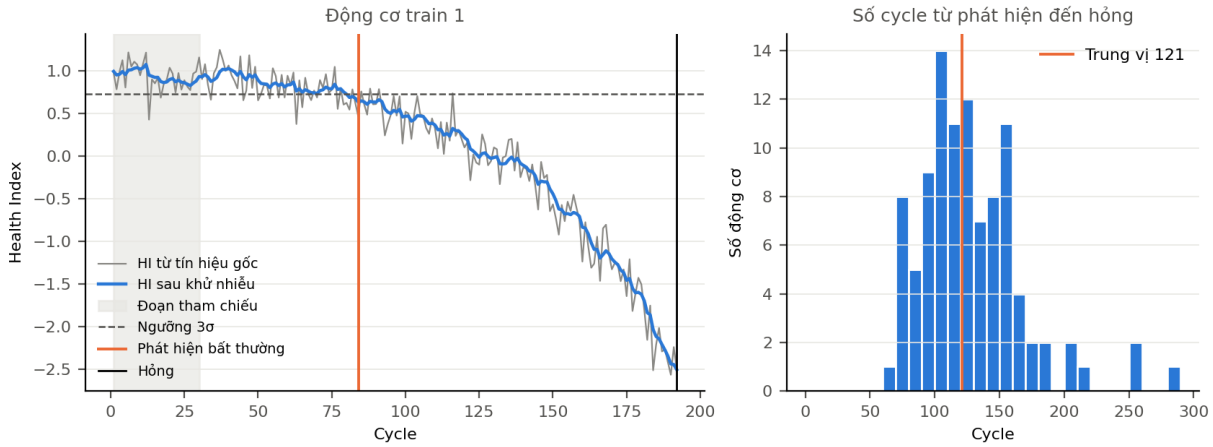
Để kiểm tra xu hướng này bằng thống kê, trung bình 30 cycles cuối được so với 30 cycles đầu của cùng động cơ. Với 15 sensor không hằng, kiểm định Wilcoxon bác bỏ giả thuyết không sau hiệu chỉnh Benjamini–Hochberg. Năm sensor 4, 7, 11, 12 và 20 thay đổi cùng hướng ở cả 100/100 động cơ (Bảng 17), tức xu hướng suy giảm rất nhất quán trong FD001. Ngược lại, `sensor_6` có ý nghĩa thống kê nhưng trung bình chênh lệch chỉ khoảng 0,00045: p-value nhỏ không đồng nghĩa với thay đổi lớn.

Bảng 17: Thay đổi tiêu biểu giữa đầu và cuối vòng đời; chênh lệch bằng trung bình 30 cycles cuối trừ trung bình 30 cycles đầu, theo đơn vị gốc của sensor.

Sensor	Trung bình chênh lệch	Khoảng tin cậy 95%	Hướng
sensor_4	20,128097	[19,402735; 20,865606]	Tăng ở 100/100 động cơ
sensor_7	−1,913527	[−1,999307; −1,829684]	Giảm ở 100/100 động cơ
sensor_11	0,613257	[0,589017; 0,637944]	Tăng ở 100/100 động cơ
sensor_12	−1,627190	[−1,704288; −1,552119]	Giảm ở 100/100 động cơ
sensor_20	−0,377007	[−0,391534; −0,363173]	Giảm ở 100/100 động cơ

Phát hiện dấu hiệu bất thường. Health Index gộp 21 sensor (14 sensor có xu hướng đã khử nhiễu, 7 sensor gần như không đổi giữ nguyên) thành một chỉ số duy nhất, bằng 0 ở mức trung bình và giảm dần khi động cơ suy giảm. Với quy tắc cố định trước (Mục 5.3: giảm quá 3 độ lệch chuẩn so với 30 cycles đầu, kéo dài 5 cycles liên tiếp), cả 100/100 động cơ train đều được phát hiện bất thường trước khi hỏng. Số cycle từ lúc phát hiện đến khi hỏng có trung vị 121, tứ phân vị 101–149,25, nhỏ nhất 67 và lớn nhất 289. Tính theo tuổi thọ, dấu hiệu bất thường xuất hiện ở khoảng 39% vòng đời (trung vị).

Hình 8 (trái) minh họa động cơ train 1: Health Index đi ngang quanh mức tham chiếu, cắt ngưỡng 3σ và được báo bất thường ở cycle 84, tức 108 cycles trước khi hỏng ở cycle 192. Hình bên phải là phân phối thời gian báo trước trên 100 động cơ.



Hình 8: Trái: Health Index của động cơ train 1 từ tín hiệu gốc (xám) và sau khử nhiễu (xanh); vùng xám là đoạn tham chiếu 30 cycles, đường đứng cam là lúc phát hiện bất thường. Phải: số cycle từ lúc phát hiện đến khi hỏng của 100 động cơ train.

Khử nhiễu ảnh hưởng trực tiếp đến kết quả này. Cùng quy tắc nhưng áp dụng trên Health Index từ tín hiệu gốc thì trung vị thời gian báo trước chỉ còn 72,5 cycles (phát hiện ở khoảng 62% vòng đời), vì độ lệch chuẩn tham chiếu lớn hơn khiến ngưỡng 3σ nằm xa hơn. Báo sớm hơn không tự động là tốt hơn: vì mọi động cơ train đều đi đến hỏng và không có nhãn thời điểm bắt đầu suy giảm, báo cáo không đo được tỷ lệ báo động giả. Đây là tín hiệu cảnh báo mức một, cho biết động cơ đã lệch khỏi trạng thái vận hành ổn định; ước lượng RUL cụ thể đến từ mô hình ở mục kế tiếp.

Độ nhạy của quy tắc phát hiện. Ngưỡng 3σ và 5 cycles liên tiếp được cố định trước khi chạy, nên cần kiểm tra kết luận có phụ thuộc mạnh vào lựa chọn này hay không. Bảng 18 chạy lại quy tắc trên Health Index sau khử nhiễu với lưới $z \in \{2, 3, 4\}$ và số cycle liên tiếp $k \in \{3, 5, 10\}$. Do không có nhãn bất thường, cột "Báo sớm" được dùng làm chỉ báo gián tiếp cho độ nhạy quá mức: số động cơ bị báo ngay trong 10 cycles sau đoạn tham chiếu (cycle 31–40), khi suy giảm thực sự khó có thể đã bắt đầu.

Bảng 18: Độ nhạy của quy tắc phát hiện bất thường trên Health Index sau EWMA, 100 động cơ train. Lead time là số cycle từ lúc phát hiện đến khi hỏng; tỷ lệ vòng đời là cycle phát hiện chia cho cycle hỏng (trung vị). Dòng in đậm là cấu hình được sử dụng.

z	k	Phát hiện	Lead time nhỏ nhất	Lead time trung vị	Tỷ lệ vòng đời	Báo sớm
2	3	100/100	78	136,0	29,6%	17
2	5	100/100	76	132,5	32,8%	8
2	10	100/100	71	124,0	37,6%	0
3	3	100/100	77	127,0	35,8%	2
3	**5**	**100/100**	**67**	**121,0**	**39,1%**	**1**
3	10	100/100	62	110,5	44,4%	0
4	3	100/100	62	114,5	42,8%	1
4	5	100/100	60	109,0	45,0%	1
4	10	100/100	50	98,5	49,2%	0

Kết luận chính không phụ thuộc vào lựa chọn tham số: cả 9 cấu hình đều phát hiện đủ 100 động cơ, và lead time nhỏ nhất luôn từ 50 cycles trở lên, lớn hơn ngưỡng cảnh báo mức hai (30 cycles). Điều thay đổi là sự đánh đổi giữa thời gian báo trước và rủi ro báo quá sớm. Cấu hình $z = 2$, $k = 3$ cho trung vị lead time lớn nhất (136 cycles) nhưng báo 17 động cơ ngay sau đoạn tham chiếu. Cấu hình $z = 4$, $k = 10$ không có trường hợp báo sớm nhưng mất khoảng 22 cycles báo trước so với cấu hình được chọn. Cấu hình $z = 3$, $k = 5$ nằm giữa hai cực: một trường hợp báo sớm và trung vị 121 cycles. Cùng lưới tham số áp dụng trên Health Index từ tín hiệu gốc cũng phát hiện đủ 100 động cơ ở mọi cấu hình nhưng trung vị lead time chỉ từ 52 đến 103 cycles (bảng đầy đủ trong `outputs/degradation/tables/anomaly_sensitivity.csv`).

6.4 Dự đoán RUL

Bảng 19 so sánh ba mô hình bằng GroupKFold 5 fold theo động cơ. HistGradientBoosting với 126 đặc trưng window (HGB) có MAE thấp nhất, 20,21 cycles, thấp hơn Ridge 3,78 cycles và thấp hơn median baseline hơn một nửa. Độ lệch chuẩn MAE giữa các fold của HGB và Ridge đều dưới 1 cycle, nhỏ hơn khoảng cách giữa hai mô hình, nên HGB được chọn làm mô hình chính.

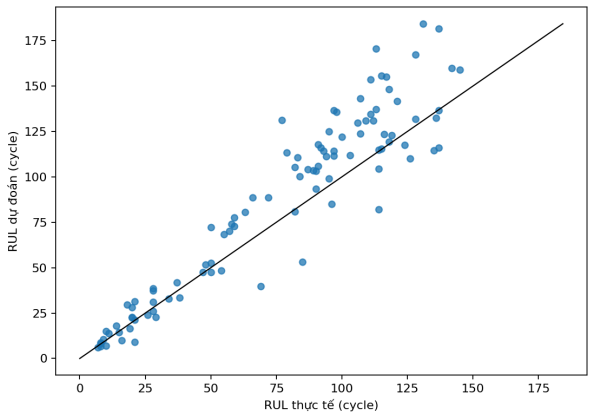
Bảng 19: Kết quả GroupKFold 5 fold trên 17.731 window train. MAE và RMSE là trung bình giữa 5 fold, tính bằng cycles.

Mô hình	MAE theo động cơ	Độ lệch chuẩn MAE	RMSE
Median baseline	46,9543	0,1193	62,9006
Ridge ($\alpha = 1$)	23,9922	0,9479	35,9467
HGB (chọn)	20,2143	0,9821	33,5479

Sau khi huấn luyện lại trên toàn bộ train, HGB đạt MAE 15,35 cycles trên 100 động cơ test (Bảng 20). Nhãn test chỉ được đọc sau khi 100 dự đoán đã được ghi ra file. MAE test thấp hơn MAE cross-validation vì hai con số đo trên các tập điểm khác nhau: cross-validation tính trên mọi window, gồm cả đầu đời có RUL lớn và khó dự đoán; test chỉ có một điểm ở cuối mỗi chuỗi bị cắt, với RUL thật từ 7 đến 145 cycles.

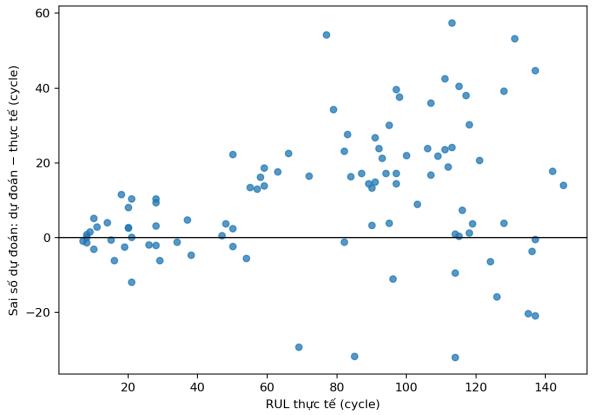
Bảng 20: Metric test của HGB trên 100 động cơ FD001; khoảng tin cậy bootstrap 95% lấy mẫu lại động cơ 10.000 lần.

Metric	Giá trị	Khoảng tin cậy 95%
MAE (cycles)	15,3490	[12,7286; 18,0866]
RMSE (cycles)	20,5413	[17,3442; 23,5850]
NASA score	1.513,2577	[743,4999; 2.476,2743]



Hình 9: RUL thật và RUL dự đoán của 100 động cơ test. Đường chéo ứng với dự đoán hoàn hảo.

Sai số phụ thuộc rõ vào RUL thật (Hình 10): MAE là 4,38 cycles với 25 động cơ test có RUL thật từ 0 đến 30, 8,77 cycles với nhóm 31–60, và khoảng 21,3–21,5 cycles với các nhóm trên 60. Mô hình dự đoán tốt khi động cơ đã gần hỏng, tức đúng vùng quan trọng cho cảnh báo, và kém hơn ở giai đoạn đầu vòng đời. Mô hình có xu hướng dự đoán cao: 74/100 dự đoán cao hơn RUL thật; mười sai số lớn nhất đều là dự đoán cao (lệch 37,6–57,5 cycles, ví dụ động cơ 23 có RUL thật 113 nhưng được dự đoán 170,5) và chiếm khoảng 75,5% tổng NASA score.



Hình 10: Sai số (dự đoán trừ thật) theo RUL thật của 100 động cơ test.

6.5 Cảnh báo sớm

Cảnh báo mức hai được phát khi RUL dự đoán ≤ 30 cycles, trên dự đoán OOF của HGB. Cả 100 động cơ train đều được cảnh báo ít nhất một lần và không động cơ nào bị bỏ lỡ. Về thời điểm, 44 động cơ được cảnh báo trước khi RUL thật chạm 30, 7 động cơ đúng lúc và 49 động cơ cảnh báo trễ (nhưng vẫn trước khi hỏng). Lead time, tức số cycle từ cảnh báo đầu tiên đến khi hỏng, có trung vị 30 cycles (Bảng 21); trường hợp trễ nhất chỉ còn 19 cycles.

Bảng 21: Phân phối lead time tại cảnh báo đầu tiên trên 100 động cơ train, ngưỡng 30, đơn vị cycle.

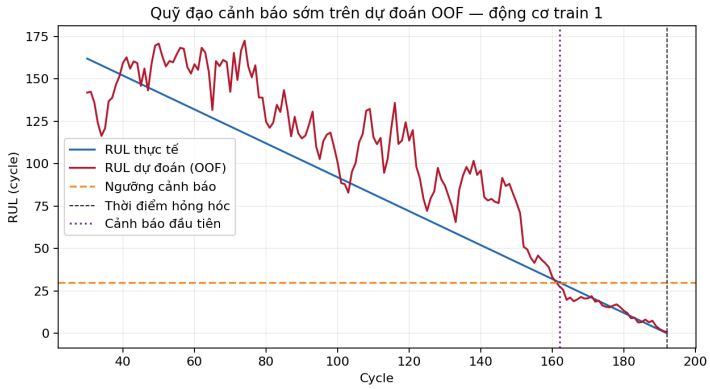
Nhỏ nhất	Q1	Trung vị	Trung bình	Q3	Lớn nhất
19	26	30	31,22	35	55

Vì gần một nửa cảnh báo bị trễ, Bảng 22 so sánh thêm ngưỡng 40 cycles trên cùng dự đoán. Ngưỡng 40 nâng tỷ lệ cảnh báo kịp thời (trước hoặc đúng lúc RUL thật chạm 30) từ 51% lên 94%, đổi lại tỷ lệ thời gian báo sớm tăng từ 1,29% lên 5,33%. Đây là phân tích độ nhạy; ngưỡng 40 chưa được hiệu chuẩn thành chính sách bảo trì.

Bảng 22: Hai ngưỡng cảnh báo trên cùng dự đoán OOF của HGB, 100 động cơ train. Kịp thời là cảnh báo đầu tiên còn ít nhất 30 cycles trước khi hỏng; báo sớm là tỷ lệ cycle bị cảnh báo khi RUL thật còn trên 30, lấy trung bình theo động cơ.

Ngưỡng	Kịp thời	Trễ/bỏ lỡ	Báo sớm (%)
30	51%	49%	1,293
40	94%	6%	5,329

Hình 11 ghép hai mức cảnh báo trên động cơ train 1. Dấu hiệu bất thường xuất hiện ở cycle 84 (Mục 6.3); RUL dự đoán chạm ngưỡng 30 ở cycle 162, đúng lúc RUL thật chạm 30, và động cơ hỏng ở cycle 192. Như vậy đội vận hành có 108 cycles để theo dõi tăng cường và 30 cycles để lên kế hoạch bảo trì.



Hình 11: RUL thật, dự đoán OOF của HGB và ngưỡng 30 cycles của động cơ train 1. Đường đứng màu tím là cảnh báo đầu tiên; đường đứng màu đen là cycle hỏng.

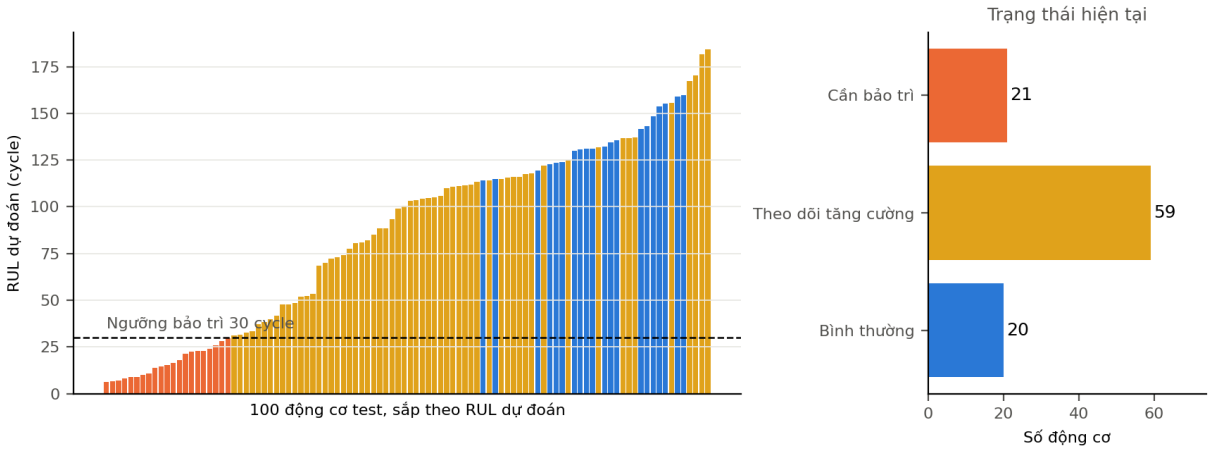
6.6 Báo cáo dự đoán thời điểm cần bảo trì

Đây là sản phẩm đặc thù của Đề tài 8: mỗi động cơ test nhận một dòng gồm cycle hiện tại, RUL dự đoán, cycle dự kiến cần bảo trì và trạng thái (Mục 5.7). Bảng 23 trích các dòng tiêu biểu trong 100 dòng; bảng đầy đủ nằm trong `outputs/maintenance/tables/maintenance_schedule.csv`.

Bảng 23: Trích lịch bảo trì dự đoán cho động cơ test, sắp theo mức ưu tiên. Cycle bảo trì là lúc RUL dự đoán chạm 30; cột bất thường là cycle Health Index bắt đầu báo bất thường (– là chưa có).

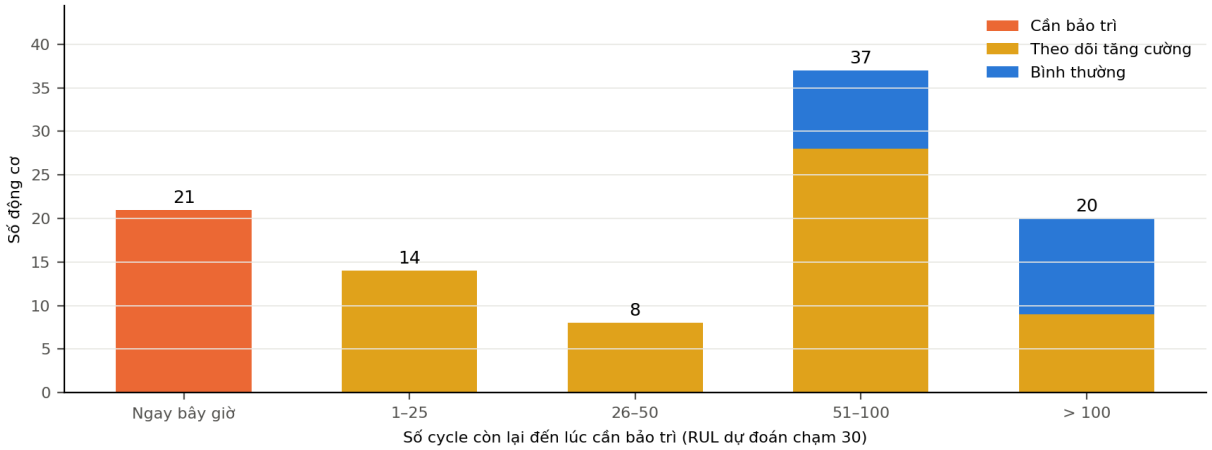
Ưu tiên	Động cơ	Cycle hiện tại	RUL dự đoán	Còn (cycle)	Cycle bảo trì	Bất thường từ cycle	Trạng thái
1	34	203	6,2	0	203	70	Cần bảo trì
2	81	213	6,7	0	213	80	Cần bảo trì
21	41	123	29,6	0	123	58	Cần bảo trì
22	90	146	31,2	1	147	67	Theo dõi tăng cường
25	91	234	33,4	3	237	59	Theo dõi tăng cường
81	11	83	114,2	84	167	–	Bình thường

Hình 12 cho thấy tình trạng toàn đội tại điểm quan sát cuối: 21 động cơ cần bảo trì ngay, 59 động cơ đã có dấu hiệu bất thường nhưng RUL dự đoán còn trên 30, và 20 động cơ bình thường. Hai động cơ (1 và 85) chỉ có 31 và 34 cycles nên chưa đủ dữ liệu để đánh giá bất thường; cả hai có RUL dự đoán trên 130 nên được xếp bình thường.



Hình 12: Trái: RUL dự đoán của 100 động cơ test, sắp tăng dần, tô màu theo trạng thái; đường đứt là ngưỡng bảo trì 30 cycles. Phải: số động cơ ở mỗi trạng thái.

Hình 13 chuyển bảng thành kế hoạch: ngoài 21 động cơ cần bảo trì ngay, có thêm 14 động cơ đến hạn trong 25 cycles tới và 8 động cơ trong khoảng 26–50 cycles. Đội bảo trì có thể dùng biểu đồ này để chuẩn bị nhân lực và phụ tùng theo từng đợt.



Hình 13: Số động cơ đến hạn bảo trì theo số cycle còn lại, tách theo trạng thái.

Sau khi lịch đã được ghi ra file, nhãn test mới được dùng để kiểm tra (Bảng 24). Trong 25 động cơ thật sự cần bảo trì (RUL thật ≤ 30), 21 động cơ được báo đúng ở mức cần bảo trì, và không động cơ nào được báo cần bảo trì khi RUL thật còn trên 30. Bốn động cơ còn lại (18, 37, 40, 90) có RUL dự đoán 31–38 nên được xếp đến hạn sau 1–8 cycles; chúng vẫn nằm ở mức theo dõi tăng cường nên không bị bỏ sót hoàn toàn. Cả 20 động cơ bình thường đều có RUL thật trên 60. Sai số tuyệt đối trung bình của số cycle còn lại đến lúc bảo trì là 14,21 cycles.

Bảng 24: Trạng thái dự đoán so với RUL thật của 100 động cơ test.

Trạng thái dự đoán	RUL thật ≤ 30	RUL thật 31–60	RUL thật > 60
Cần bảo trì	21	0	0
Theo dõi tăng cường	4	14	41
Bình thường	0	0	20

Mức theo dõi tăng cường còn rộng: 41/59 động cơ ở mức này có RUL thật trên 60. Lý do là ngưỡng 3σ của Health Index phát hiện suy giảm từ khoảng 39% vòng đời, nên mức này phù hợp để tăng tần suất kiểm tra, chưa phải để lên lịch can thiệp. Ngược lại, lịch cũng có thể quá lạc quan với từng động cơ: động cơ 23 được dự đoán cần bảo trì ở cycle 270, trong khi RUL thật 113 cho thấy thời điểm đó vào khoảng cycle 213 (Mục 6.4). Tính trên cả 100 động cơ, 12 động cơ có cycle bảo trì dự kiến muộn hơn thời điểm hỏng thật từ 4 đến 27 cycles (7 động cơ ở mức theo dõi tăng cường, 5 động cơ ở mức bình thường). Cả 12 động cơ đều có RUL thật từ 77 cycles trở lên, tức là lỗi này xảy ra ở các động cơ còn xa thời điểm hỏng, nơi mô hình dự đoán dư RUL (Mục 6.4). Trong vận hành, lịch được tính lại mỗi khi có dữ liệu mới, nên với các động cơ này lịch sẽ được cập nhật nhiều lần trước khi đến hạn; tuy nhiên kết quả cho thấy không nên chốt lịch bảo trì xa chỉ từ một lần dự đoán.

6.7 Thử nghiệm mở rộng

Ngoài luồng chính, một số thử nghiệm được chạy để hiểu giới hạn của mô hình. Chúng không thay đổi mô hình hay chính sách cảnh báo ở trên; chi tiết và artifact được lưu trong `experiments/` và `outputs/experiments/`.

Cải thiện mô hình. Nested cross-validation thử window 10/20/30, 24 cấu hình siêu tham số và ba phương án tăng trọng số cho vùng RUL thấp; quy trình chọn trên toàn bộ train quay về đúng cấu hình gốc, nên dự đoán test không đổi. Thêm cột `cycle` và 42 đặc trưng lịch sử (169 đặc trưng) giảm MAE OOF từ 20,21 xuống 17,00 cycles trên cùng các fold, nhưng làm tỷ lệ cảnh báo kịp thời ở ngưỡng 30 giảm từ 51% xuống 45% và chưa được kiểm chứng bằng một lần đánh giá độc lập, nên chưa thay mô hình chính. Similarity-based RUL thua HGB ở cả 3/3 fold và bị dừng.

Chuyển sang FD002–FD004. Với window 10 cycles, mô hình huấn luyện trên FD001 có MAE 47,39–60,32 cycles khi áp dụng trực tiếp lên ba subset còn lại, so với 22,36–28,33 cycles khi mỗi subset tự huấn luyện, tức tăng 1,7–2,7 lần (Bảng 25). Nguyên nhân chính là điều kiện vận hành khác nhau (Mục 7.3).

Bảng 25: MAE (cycles) khi huấn luyện nội bộ và khi chuyển mô hình FD001, window 10; khoảng tin cậy 95% lấy mẫu lại động cơ 10.000 lần.

Subset test	Huấn luyện nội bộ	Mô hình FD001
FD002	22,36 [19,71; 25,05]	60,32 [56,05; 64,80]
FD003	28,33 [22,47; 34,55]	47,39 [40,63; 54,35]
FD004	27,22 [24,18; 30,37]	58,57 [53,93; 63,30]

Dự báo Health Index bằng ARIMA. Trên 10 động cơ chọn trước, ARIMA dự báo 30% cuối chuỗi Health Index với MAE trung bình 0,4110, thấp hơn hai baseline `drift` (0,7469) và

`last_value` (0,9478). Tuy vậy ARIMA chỉ tốt hơn `drift` ở 7/10 động cơ và khoảng dự đoán 95% chỉ bao phủ 21–97% điểm thật, nên đây là phân tích mô tả, không dùng để cảnh báo.

7 Thảo luận

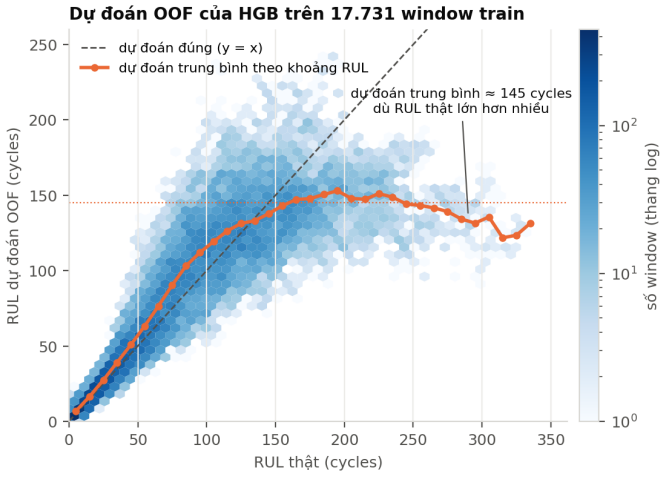
Mục này giải thích các kết quả ở Mục 6: sai số của mô hình RUL đến từ đâu, vì sao một phần cảnh báo bị trễ và hai mức cảnh báo bổ sung cho nhau thế nào. Các số liệu chẩn đoán được tính lại từ artifact đã có bằng `scripts/report_error_diagnostics.py`, không huấn luyện lại mô hình.

7.1 Sai số của mô hình RUL đến từ đâu

Bảng 26 và Hình 14 chia sai số OOF của HGB trên train theo RUL thật. Mô hình dự đoán tốt ở cuối vòng đời: MAE chỉ 3,67 cycles khi RUL thật không quá 30. Sai số tăng dần khi RUL lớn hơn, và thay đổi hẳn tính chất khi RUL vượt khoảng 150 cycles. Ở vùng này, dự đoán trung bình gần như không đổi, quanh 145–147 cycles, bất kể RUL thật là 160 hay 300.

Bảng 26: Sai số OOF của HGB trên 17.731 window train, chia theo RUL thật. Sai số có dấu bằng dự đoán trừ thật.

RUL thật (cycles)	Số window	MAE	Sai số có dấu trung bình	Dự đoán trung bình
0–30	3.100	3,67	+1,47	16,47
31–60	3.000	10,68	+5,59	51,09
61–90	3.000	20,85	+14,52	90,02
91–125	3.392	22,81	+12,88	120,61
126–150	1.999	20,82	−2,87	134,73
151–200	2.226	30,25	−23,30	147,16
201–361	1.014	91,29	−91,29	145,36



Hình 14: RUL dự đoán OOF so với RUL thật. Đường cam là dự đoán trung bình theo từng khoảng 10 cycles: bám sát đường chéo khi RUL nhỏ, nhưng dừng quanh 145 cycles khi RUL thật lớn hơn khoảng 150.

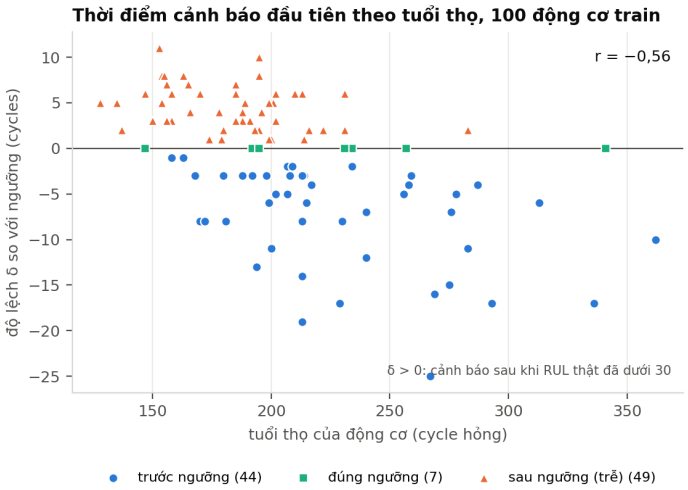
Hiện tượng này khớp với kết quả phát hiện bất thường ở Mục 6.3: Health Index đi ngang trong khoảng 39% đầu vòng đời (trung vị) rồi mới giảm rõ. Ở giai đoạn vận hành ổn định,

sensor chủ yếu phản ánh dao động đo và khác biệt ban đầu giữa các động cơ, nên khó phân biệt một động cơ còn 160 cycles với một động cơ còn 300 cycles. Điều này cũng giải thích vì sao MAE cross-validation (20,21 cycles) cao hơn MAE test (15,35 cycles): 29,5% số window train có RUL thật trên 125, trong khi RUL thật của test chỉ từ 7 đến 145.

Ở vùng giữa (RUL 61–125), mô hình có xu hướng dự đoán cao hơn thật khoảng 13–15 cycles. Mười sai số lớn nhất trên test rơi vào các động cơ có RUL thật 77–137 và đều là dự đoán cao, nên sai số test lặp lại đúng xu hướng đã thấy trên OOF. So với nghiên cứu liên quan, Li và cộng sự báo cáo RMSE 12,61 cycles trên FD001 với CNN khi cắt nhãn ở 125 cycles [9]; RMSE của báo cáo là 20,54 cycles. Hai kết quả không so trực tiếp được vì khác số sensor, cách chuẩn hóa và mô hình, nhưng khoảng cách gợi ý rằng sáu thống kê trên window có thể làm mất một phần thông tin về hình dạng chuỗi.

7.2 Vì sao gần một nửa cảnh báo bị trễ

Cảnh báo trễ không phân bố đều giữa các động cơ. 49 động cơ bị cảnh báo sau ngưỡng có tuổi thọ trung bình 182,8 cycles, trong khi 44 động cơ được cảnh báo trước ngưỡng có tuổi thọ trung bình 229,1 cycles. Hệ số tương quan giữa tuổi thọ và độ lệch cảnh báo δ_i là $-0{,}56$: động cơ càng ngắn đời thì càng bị cảnh báo muộn (Hình 15).



Hình 15: Độ lệch của cảnh báo đầu tiên so với ngưỡng theo tuổi thọ của 100 động cơ train. Cảnh báo trễ (tam giác cam) tập trung ở các động cơ hỏng trước khoảng cycle 230.

Mô hình học một quan hệ trung bình giữa mức sensor và RUL. Động cơ ngắn đời suy giảm nhanh hơn trung bình, nên tại cùng một mức sensor, RUL thật của chúng thấp hơn điều mô hình kỳ vọng và RUL dự đoán chạm ngưỡng 30 muộn hơn thực tế. Đây là loại lỗi nguy hiểm nhất: động cơ hỏng nhanh nhất lại nhận cảnh báo muộn nhất. Ngưỡng 40 giảm tỷ lệ trễ từ 49% xuống 6% với chi phí báo sớm nhiều hơn khoảng bốn lần (Bảng 22); chọn ngưỡng cụ thể cần chi phí bảo trì và downtime, mà dữ liệu không có.

Hai mức cảnh báo bổ sung cho nhau. Mức một (bất thường trên Health Index) không cần nhãn và báo rất sớm, trung vị 121 cycles trước khi hỏng, nhưng chỉ cho biết động cơ đã lệch

khỏi trạng thái vận hành ổn định, không ước lượng thời gian còn lại. Mức hai (RUL dự đoán ≤ 30) cho con số cụ thể để lên kế hoạch, nhưng có thể trễ với động cơ suy giảm nhanh. Trong vận hành, mức một có thể dùng để tăng tần suất theo dõi, còn mức hai dùng để đặt lịch bảo trì. Bảng 24 xác nhận cách chia này trên test: mức hai không có trường hợp dương tính giả, còn mức một rộng hơn (59 động cơ) và vẫn giữ được 4 động cơ mà mức hai bỏ qua.

7.3 Vì sao mô hình FD001 không chuyển được sang subset khác

Với FD002 và FD004, chỉ khoảng 14–15% giá trị test của 9 sensor có xu hướng nằm trong khoảng min–max của train FD001, do sáu điều kiện vận hành đẩy mức sensor ra xa điều kiện duy nhất của FD001. Mô hình cây không ngoại suy được, nên dự đoán gần như hằng số (độ lệch chuẩn khoảng 13 cycles) và cao hơn RUL thật khoảng 50 cycles. Với FD003, giá trị sensor nằm trong miền FD001 (94,7%) nhưng dự đoán vẫn cao hơn thật trung bình 45 cycles; một giả thuyết hợp lý là dạng lỗi fan của FD003 làm sensor thay đổi khác với suy giảm HPC mà mô hình đã học. Hai kiểu thất bại này cần hai cách xử lý: chuẩn hóa theo điều kiện vận hành cho FD002/FD004, và dữ liệu có dạng lỗi tương ứng cho FD003.

7.4 Ý nghĩa đối với xử lý dữ liệu lớn

Với 265.256 quan sát trên cả bốn subset, dữ liệu C-MAPSS vẫn vừa trong bộ nhớ của một máy, nên Pandas đủ dùng. Giá trị của phần dữ liệu lớn trong báo cáo nằm ở cách tổ chức: kiểm tra contract trước khi xử lý, lưu dữ liệu theo lớp, và có một implementation PySpark cho kết quả giống hệt Pandas. Khi dữ liệu tăng lên (nhiều đội động cơ, tần suất lấy mẫu cao hơn), phần ETL và rolling features có thể chuyển sang Spark với bằng chứng parity sẵn có; benchmark $10\times$ ở Bảng 14 cho thấy Spark đã nhanh hơn Pandas khoảng 3,4 lần ngay trên một máy.

Bảng 27 đối chiếu năm khía cạnh của bài toán với bằng chứng thực tế và giới hạn tương ứng.

Bảng 27: Đối chiếu 5V, bằng chứng và giới hạn của pipeline.

Khía cạnh	Bằng chứng trong báo cáo	Giới hạn hiện tại
Volume	265.256 quan sát từ bốn subset; dữ liệu raw khoảng 46 MB; lưu trên HDFS; benchmark nhân bản $10\times$.	Vẫn vừa bộ nhớ một máy; dữ liệu nhân bản chỉ đo chi phí kỹ thuật.
Velocity	Pipeline batch theo từng subset; khử nhiễu, bất thường và cảnh báo đều tính nhân quả theo cycle.	Chưa có streaming hoặc yêu cầu độ trễ thời gian thực.
Variety	21 sensor, 3 operational settings, chuỗi run-to-failure, nhãn RUL; lưu qua text, Parquet, SQLite.	Cùng một nguồn cảm biến dạng bảng; chưa có dữ liệu phi cấu trúc.
Veracity	Manifest MD5/SHA-256 của nguồn, contract schema, kiểm tra khóa và cycle, giá trị thiếu, parity Pandas–PySpark–MapReduce, khử nhiễu EWMA.	Chưa kiểm chứng với lỗi cảm biến và dữ liệu thiếu của hệ thống thực.
Value	Phát hiện bất thường, dự đoán RUL và hai mức cảnh báo sớm.	Chưa có chi phí bảo trì, downtime hoặc KPI kinh tế.

7.5 Giới hạn

Về dữ liệu. C-MAPSS là dữ liệu mô phỏng; động cơ thực tế có lỗi cảm biến, dữ liệu thiếu và điều kiện vận hành phức tạp hơn. Dữ liệu không có chi phí bảo trì hay cách quy đổi cycle sang thời gian thực, nên lịch bảo trì ở Mục 6.6 chỉ tính bằng cycle với biên an toàn cố định 30 cycles, chưa tối ưu theo chi phí.

Về đánh giá. Metric test chỉ đo tại một điểm cắt mỗi động cơ. Kết quả cảnh báo dùng lại tập OOF đã dùng để chọn mô hình. Phát hiện bất thường không đo được trực tiếp tỷ lệ báo động giả vì không có động cơ vận hành bình thường đến hết chuỗi quan sát và không có nhãn thời điểm bắt đầu suy giảm; phân tích độ nhạy (Bảng 18) chỉ cho thấy kết luận ổn định theo tham số, không thay thế được phép đo này. EWMA được đánh giá bằng mức giảm dao động, không bằng tín hiệu sạch.

Về hạ tầng. Phần ETL chính dùng Spark `local[4]` trên Windows và ghi qua driver. HDFS, YARN và MapReduce chạy trên cụm Hadoop một node trong Docker, đủ để kiểm chứng tính đúng của phép tính nhưng chưa đo được hiệu năng phân tán trên nhiều máy. Dữ liệu nhân bản không làm tăng số mẫu khoa học độc lập.

7.6 Hướng phát triển

Ba hướng có giá trị nhất là: (1) chọn ngưỡng cảnh báo theo chi phí bảo trì khi có dữ liệu chi phí, hoặc theo mức lead time tối thiểu với xác suất cho trước; (2) đưa tín hiệu bất thường mức một vào đặc trưng của mô hình RUL để giảm cảnh báo trễ ở động cơ suy giảm nhanh; (3) chạy pipeline Spark trên cụm nhiều máy với dữ liệu streaming để kiểm chứng phần Velocity.

8 Khó khăn và bài học rút ra

Khó khăn 1: dữ liệu không có nhãn bất thường. Đề tài yêu cầu phát hiện dấu hiệu bất thường, nhưng C-MAPSS không ghi lúc nào động cơ bắt đầu hỏng. Cách xử lý là so sánh mỗi động cơ với chính nó trong 30 cycles đầu, giai đoạn vận hành ổn định, thay vì dựa vào nhãn. Bài học: khi thiếu nhãn, cần xác định rõ trạng thái tham chiếu trước khi đo độ lệch, đồng thời nêu rõ đại lượng chưa kiểm chứng được (ở đây là tỷ lệ dương tính giả).

Khó khăn 2: rò rỉ dữ liệu khi chia tập. Nếu chia ngẫu nhiên theo dòng, các window gần như trùng nhau của cùng một động cơ sẽ nằm ở cả train và validation, làm sai số validation thấp hơn thực tế. Cách xử lý là chia GroupKFold theo động cơ, tính window và EWMA chỉ từ quá khứ, và chỉ đọc nhãn test sau khi đã ghi dự đoán. Bài học: với dữ liệu chuỗi thời gian, đơn vị chia tập phải là thực thể (động cơ), không phải dòng.

Khó khăn 3: Spark chạy trên Windows. PySpark cần Java, và trên Windows thiếu Hadoop native (`winutils.exe`) nên không ghi file trực tiếp được. Cách xử lý là dùng JRE portable, ghi qua Arrow từ driver và kiểm tra parity với Pandas sau mỗi bước. Bài học: khi có hai implementation, phải đối chiếu từng giá trị chứ không chỉ số dòng; parity bằng 0 là bằng chứng mạnh hơn mọi lời mô tả.

Khó khăn 4: dựng Hadoop trong Docker. Image `apache/hadoop:3.3.6` không có Python nên job Hadoop Streaming viết bằng Python không chạy được; cách xử lý là chuyển sang image 3.4.3. Lần chạy đầu, Spark client không đọc cấu hình của cụm nên ghi file với replication 3 trong khi chỉ có một DataNode, và `hdfs fsck` báo các block thiếu bản sao; cách xử lý là truyền `dfs.replication=1` cho Spark. Bài học: job báo thành công chưa đủ, cần kiểm tra trạng thái HDFS bằng `fsck` sau khi ghi.

Khó khăn 5: dễ sa vào tối ưu mô hình. Nhiều vòng thử cải thiện RUL (window, siêu tham số, trọng số, thêm đặc trưng lịch sử, similarity) tốn nhiều thời gian nhưng không đổi kết luận chính. Bài học: với bài toán bảo trì dự đoán, một pipeline dữ liệu đúng, cảnh báo dễ hiểu và giới hạn được nói rõ có giá trị hơn vài cycle MAE.

9 Kết luận

Báo cáo đã xây dựng một pipeline bảo trì dự đoán tái lập được trên dữ liệu cảm biến NASA C-MAPSS FD001, đi từ dữ liệu thô qua các lớp Bronze–Silver–Gold đến phát hiện bất thường, dự đoán RUL và cảnh báo sớm. Các câu hỏi được trả lời như sau:

Q1. Dữ liệu FD001 (20.631 dòng train, 13.096 dòng test) đạt mọi kiểm tra về schema, khóa và tính liên tục của cycle. ETL bằng PySpark khớp hoàn toàn với Pandas trên ba bảng Silver/Gold; ở scale $10\times$, Spark local nhanh hơn Pandas khoảng 3,4 lần. Trên cụm Hadoop một node, MapReduce và Spark đọc dữ liệu từ HDFS cũng cho kết quả khớp tuyệt đối với Pandas.

Q2. EWMA span 10 giảm khoảng 86% dao động ngắn hạn của 14 sensor có thông tin mà vẫn giữ xu hướng suy giảm; window 30 cycles tổng hợp 21 sensor thành 126 đặc trưng.

Q3. 15/21 sensor thay đổi có ý nghĩa thống kê giữa đầu và cuối vòng đời; năm sensor 4, 7, 11, 12 và 20 thay đổi cùng hướng ở cả 100 động cơ. Health Index từ tín hiệu đã khử nhiễu phát hiện bất thường ở cả 100/100 động cơ train, trung vị 121 cycles trước khi hỏng.

Q4. HistGradientBoosting đạt MAE 15,35 cycles (khoảng tin cậy 95%: 12,73–18,09) và RMSE 20,54 cycles trên 100 động cơ test; sai số nhỏ nhất ở vùng gần hỏng (MAE 4,38 cycles khi RUL thật ≤ 30).

Q5. Với ngưỡng RUL dự đoán ≤ 30, không động cơ nào bị bỏ lỡ, lead time trung vị 30 cycles, nhưng 49% cảnh báo bị trễ, tập trung ở động cơ suy giảm nhanh. Ngưỡng 40 giảm tỷ lệ trễ xuống 6% với thời gian báo sớm tăng khoảng bốn lần. Lịch bảo trì cho 100 động cơ test báo cần bảo trì ngay 21 động cơ, đúng cả 21; 4 động cơ còn lại thật sự cần bảo trì được xếp đến hạn sau 1–8 cycles.

Sản phẩm chính của báo cáo là lịch dự đoán thời điểm cần bảo trì cho từng động cơ, dựa trên hai mức cảnh báo bổ sung cho nhau: mức một báo động cơ đã lệch khỏi trạng thái vận hành ổn định, mức hai báo thời điểm cần bảo trì; cả hai được thể hiện trên biểu đồ xu hướng và biểu đồ kế hoạch bảo trì. Các thử nghiệm mở rộng cho thấy mô hình FD001 không dùng trực tiếp được cho điều kiện vận hành khác, và các hướng tinh chỉnh mô hình chưa làm thay đổi kết luận chính. Kết quả chỉ áp dụng cho dữ liệu mô phỏng; việc chọn ngưỡng cảnh báo thực tế cần thêm dữ liệu chi phí bảo trì.

Tài liệu tham khảo

[1] Apache Software Foundation. Apache Parquet Documentation: Overview. Apache Parquet documentation. Accessed 2026-10-07. URL: https://parquet.apache.org/docs/overview/.

[2] Apache Software Foundation. Hadoop Streaming. Apache Hadoop documentation. Accessed 2026-10-09; cluster uses Hadoop 3.4.3. URL: https://hadoop.apache.org/docs/stable/hadoop-streaming/HadoopStreaming.html.

[3] Apache Software Foundation. Spark SQL and DataFrames. Apache Spark documentation. Accessed 2026-09-22; implementation version not selected. URL: https://spark.apache.org/docs/latest/sql-programming-guide.html.

[4] Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological)*, 57(1):289–300, 1995. doi:10.1111/j.2517-6161.1995.tb02031.x.

[5] Jeffrey Dean and Sanjay Ghemawat. MapReduce: Simplified data processing on large clusters. *Communications of the ACM*, 51(1):107–113, 2008. doi:10.1145/1327452.1327492.

[6] B. Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi:10.1214/aos/1176344552.

[7] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, Melbourne, Australia, 3 edition, 2021. Online edition, sections 5.4, 8.1, 9.1, 9.5 and 9.7; accessed 2026-09-22. URL: https://otexts.com/fpp3/.

[8] Yaguo Lei, Naipeng Li, Liang Guo, Ningbo Li, Tao Yan, and Jing Lin. Machinery health prognostics: A systematic review from data acquisition to RUL prediction. *Mechanical Systems and Signal Processing*, 104:799–834, 2018. URL: https://www.sciencedirect.com/science/article/pii/S0888327017305988, doi:10.1016/j.ymssp.2017.11.016.

[9] Xiang Li, Qian Ding, and Jian-Qiao Sun. Remaining useful life estimation in prognostics using deep convolution neural networks. *Reliability Engineering & System Safety*, 172:1–11, 2018. doi:10.1016/j.ress.2017.11.021.

[10] NASA. CMAPSS jet engine simulated data. NASA Open Data Portal. Accessed 2026-09-20; project uses FD001. URL: https://data.nasa.gov/dataset/cmapss-jet-engine-simulated-data.

[11] Emmanuel Ramasso and Abhinav Saxena. Review and analysis of algorithmic approaches developed for prognostics on CMAPSS dataset. In *Annual Conference of the PHM Society*, volume 6, 2014. Issue 1. URL: https://papers.phmsociety.org/index.php/phmconf/article/view/2512, doi:10.36001/phmconf.2014.v6i1.2512.

[12] Abhinav Saxena, Jose Celaya, Bhaskar Saha, Sankalita Saha, and Kai Goebel. Metrics for offline evaluation of prognostic performance. *International Journal of Prognostics and Health Management*, 1(1), 2010. URL: https://papers.phmsociety.org/index.php/ijphm/article/view/1336, doi:10.36001/ijphm.2010.v1i1.1336.

[13] Abhinav Saxena, Kai Goebel, Don Simon, and Neil Eklund. Damage propagation modeling for aircraft engine run-to-failure simulation. In *2008 International Conference on Prognostics and Health Management*, pages 1–9. IEEE, 2008. doi:10.1109/PHM.2008.4711414.

[14] scikit-learn developers. Cross-validation iterators for grouped data. User Guide. Accessed 2026-09-22; implementation version not selected. URL: https://scikit-learn.org/stable/modules/cross_validation.html#cross-validation-iterators-for-grouped-data.

[15] scikit-learn developers. Decomposing signals in components: Principal Component Analysis. User Guide. Accessed 2026-09-22; implementation version not selected. URL: https://scikit-learn.org/stable/modules/decomposition.html#pca.

[16] scikit-learn developers. Nested versus non-nested cross-validation. scikit-learn 1.9.1 documentation. Accessed 2026-09-27; documentation version 1.9.1. URL: https://scikit-learn.org/stable/auto_examples/model_selection/plot_nested_cross_validation_iris.html.

[17] SciPy developers. scipy.stats.bootstrap. SciPy 1.16.2 Manual. Accessed 2026-09-22. URL: https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.stats.bootstrap.html.

[18] SciPy developers. scipy.stats.false_discovery_control. SciPy 1.16.2 Manual. Accessed 2026-09-22. URL: https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.stats.false_discovery_control.html.

[19] SciPy developers. scipy.stats.wilcoxon. SciPy 1.16.2 Manual. Accessed 2026-09-22. URL: https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.stats.wilcoxon.html.

[20] Konstantin Shvachko, Hairong Kuang, Sanjay Radia, and Robert Chansler. The Hadoop distributed file system. In *2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)*, pages 1–10, 2010. doi:10.1109/MSST.2010.5496972.

[21] statsmodels developers. statsmodels.tsa.stattools.adfuller. Online documentation. Accessed 2026-09-22; implementation version not selected. URL: https://www.statsmodels.org/stable/generated/statsmodels.tsa.stattools.adfuller.html.

[22] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945. doi:10.2307/3001968.

[23] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, pages 15–28. USENIX Association, 2012. URL: https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia.

[24] Shuai Zheng, Kosta Ristovski, Ahmed Farahat, and Chetan Gupta. Long short-term memory network for remaining useful life estimation. In *2017 IEEE International Conference on Prognostics and Health Management (ICPHM)*, pages 88–95. IEEE, 2017. doi:10.1109/ICPHM.2017.7998311.